Nama: Raven Kongnando Lasher

NIM: 2602624903

Link:

https://drive.google.com/drive/folders/1a1YVeCqLgG-vcBxh114vHLJAeVCUHt3B

Ujian Machine Learning

1. Silahkan gunakan dataset public yang bisa digunakan untuk klasifikasi. Adapun dataset tersebut:
   - Jumlah kelasnya minimal 3
   - Jumlah fiturnya (selain kelas minimal 7)
   - Jumlah total datanya minimal 1000.

   Pertanyaan:

   a. Tunjukkan data linknya dan lakukan EDA (Exploratory Data Analysis) dari dataset yang digunakan **(LO3) (5 point)**
   Dataset: Wine Quality – Red Wine
   Link Dataset: https://www.kaggle.com/datasets/yasserh/wine-quality-dataset
   Link ipynb :
   https://colab.research.google.com/drive/1gkocpAXXL9LIz3dBN-LmiEPrVErcmowh?usp=sharing

   Spesifikasi Dataset:
   - Jumlah data: 1143 sampel
   - Jumlah fitur: 11 fitur numerik (tanpa kolom Id)
   - Target: quality (nilai integer dari 3–8 → 6 kelas)

   Hasil EDA:
   - Struktur Data
   Dataset memiiliki 13 Kolom yang terdiri dari 11 Fitur + 1 target + 1 id. Tidak ada nilai kosong

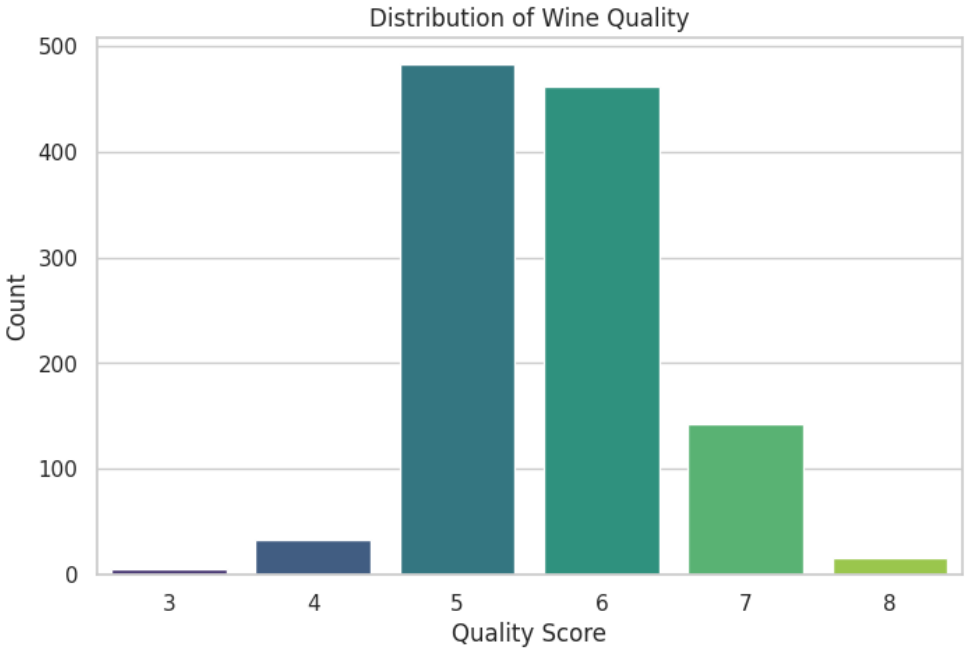
```
# Load the dataset
df = pd.read_csv("WineQT.csv")
df.drop(columns=['Id'], inplace=True)

# Preview the first few rows
df.head()
```

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	alcohol	quality
0	7.4	0.70	0.00	1.9	0.076	11.0	34.0	0.9978	3.51	0.56	9.4	5
1	7.8	0.88	0.00	2.6	0.098	25.0	67.0	0.9968	3.20	0.68	9.8	5
2	7.8	0.76	0.04	2.3	0.092	15.0	54.0	0.9970	3.26	0.65	9.8	5
3	11.2	0.28	0.56	1.9	0.075	17.0	60.0	0.9980	3.16	0.58	9.8	6
4	7.4	0.70	0.00	1.9	0.076	11.0	34.0	0.9978	3.51	0.56	9.4	5

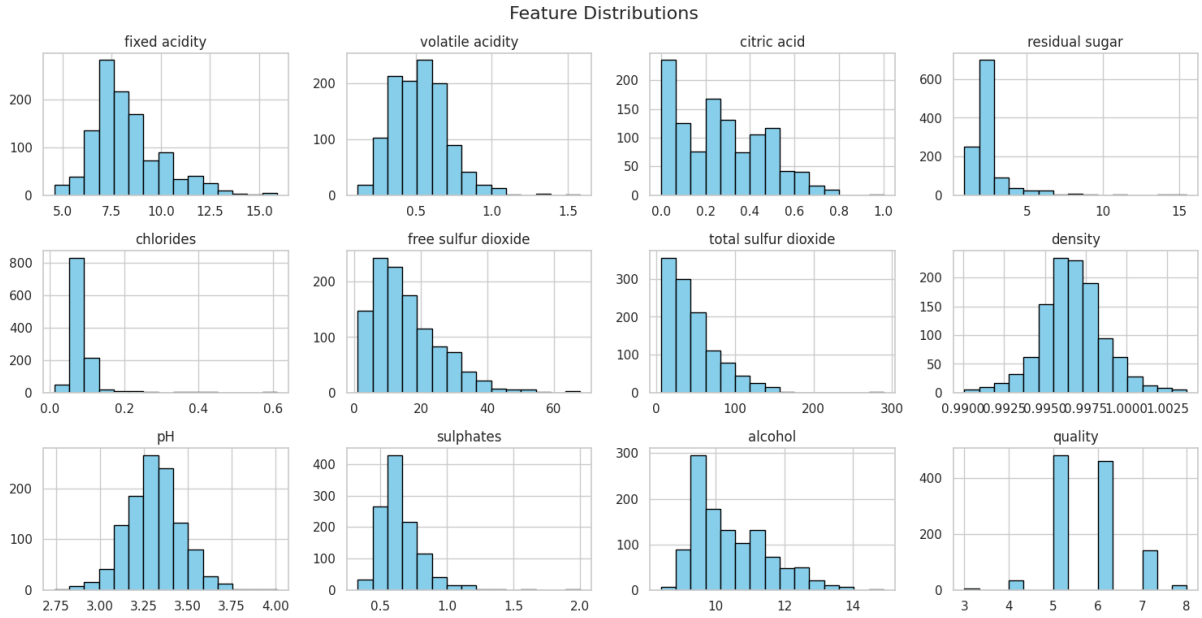
```
[ ]  # Check basic info
     print("\nData Information:")
     df.info()
```

```
Data Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1143 entries, 0 to 1142
Data columns (total 12 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   fixed acidity         1143 non-null   float64
 1   volatile acidity      1143 non-null   float64
 2   citric acid           1143 non-null   float64
 3   residual sugar        1143 non-null   float64
 4   chlorides             1143 non-null   float64
 5   free sulfur dioxide   1143 non-null   float64
 6   total sulfur dioxide  1143 non-null   float64
 7   density               1143 non-null   float64
 8   pH                    1143 non-null   float64
 9   sulphates             1143 non-null   float64
 10  alcohol               1143 non-null   float64
 11  quality               1143 non-null   int64
dtypes: float64(11), int64(1)
memory usage: 107.3 KB
```

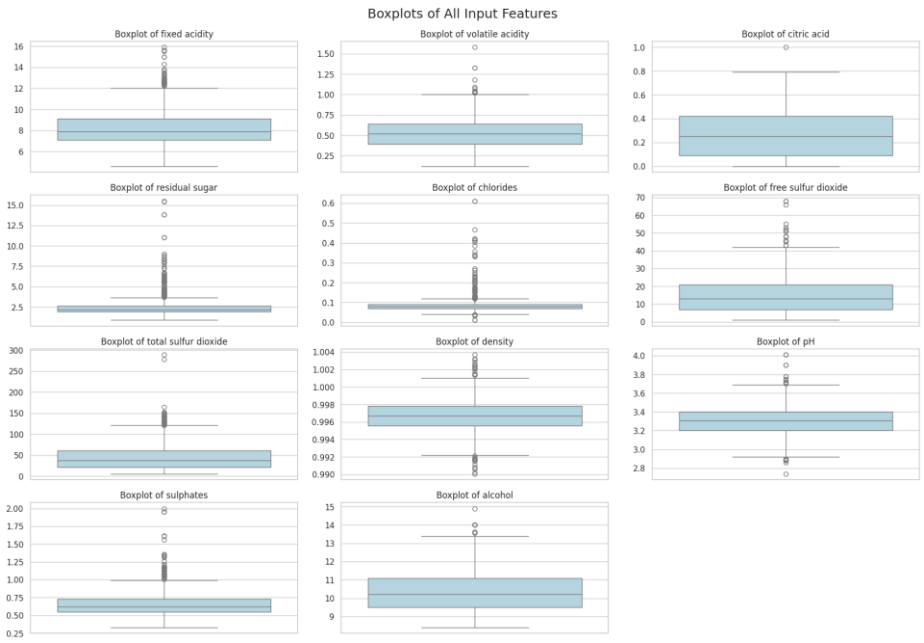
- Distribusi Target (quality)
  Distribusi kelas tidak seimbang: mayoritas adalah kelas 5 dan 6.



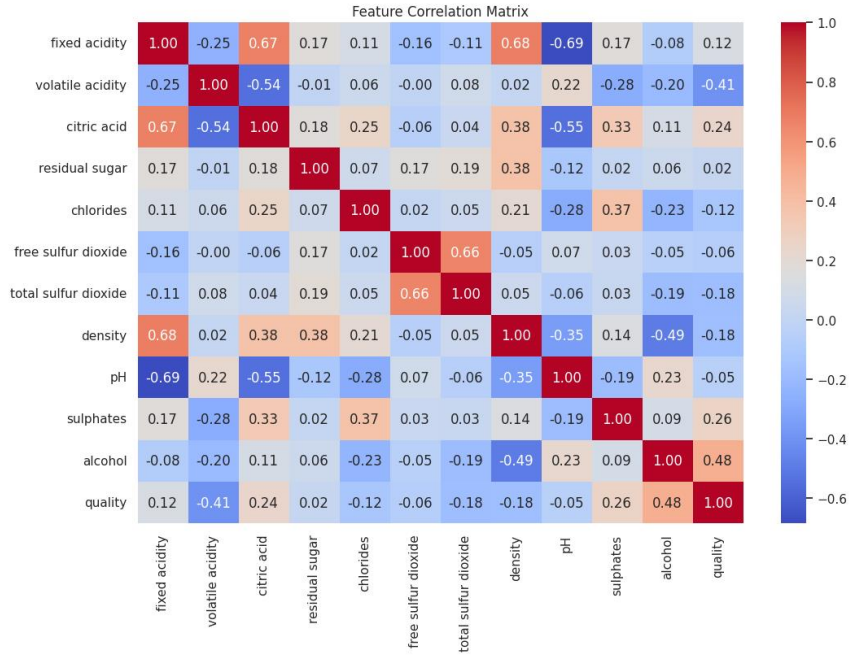
- Distribusi Fitur
  Beberapa fitur seperti residual sugar, sulphates, dan alcohol memiliki skewness tinggi.



- Boxplot Fitur
  Beberapa fitur memiliki potensi outlier terutama residual_sugar, chlorides, dan juga total sulfur tapi kita tidak akan menghapus karena datasets tersebut berdasarkan real-world.



- Korelasi Antar Fitur dan Target
  Korelasi positif kuat antara alcohol dan quality.
  Korelasi negatif antara volatile acidity dan quality.



Kesimpulan EDA:
Dataset bersih dan siap diguakan. Namun, ketidakseimbangan kelas perlu diperhatikan saat klasifikasi agar tidak terjadi bias terhadap kelas mayoritas. Sehingga mungkin bakal ada percobaan SMOTE untuk masalah teresebut.

b. Lakukan klasifikasi dengan SVM, KNN, dan ANN. Lakukan pembagian data training dan testing. Tunjukan hasil performancenya (multi class) dan lakukan analisis **(LO1, LO2, LO4) (20 point)**

Tahapan kerja:

1. **Import data dan eksplore dataset / EDA Sudah dibahas di A**
2. **Preprocess data:**

- Melakukan scalng pada fitur data
- Split data: 80% pelatihan, 20% pengujian, dengan stratified sampling

```
[ ]  from sklearn.model_selection import train_test_split
     from sklearn.preprocessing import StandardScaler

     # Pisahkan fitur dan target
     X = df.drop('quality', axis=1)
     y = df['quality']
```

```
[ ]  # Scaling
     scaler = StandardScaler()
     X_scaled = scaler.fit_transform(X)
```

```
[ ]  # Stratified train-test split (80:20)
     X_train, X_test, y_train, y_test = train_test_split(
         X_scaled, y, test_size=0.2, stratify=y, random_state=42
     )
```

```
     # Cek distribusi kelas di train dan test
     print("Train set class distribution:\n", y_train.value_counts().sort_index())
     print("\nTest set class distribution:\n", y_test.value_counts().sort_index())
```

3. Training data:

Terdapat 2 eksperimen untuk 3 model (SVM, KNN, ANN)

- Tanpa balancing / SMOTE

```
# SVM Model
print("\n===== SVM Classifier =====")
svm_model = SVC(kernel='rbf', C=1, gamma='scale')
svm_model.fit(X_train, y_train)
```

```
# KNN Model
print("\n===== KNN Classifier =====")
knn_model = KNeighborsClassifier(n_neighbors=5)
knn_model.fit(X_train, y_train)
```

```
# ANN Model
print("\n===== ANN Classifier =====")

# One-hot encode target for ANN
y_train_ann = to_categorical(y_train)
y_test_ann = to_categorical(y_test)

# Get number of features and classes
n_features = X_train.shape[1]
n_classes = y_train_ann.shape[1]

# Build model
ann_model = Sequential()
ann_model.add(Dense(64, activation='relu', input_shape=(n_features,)))
ann_model.add(Dense(32, activation='relu'))
ann_model.add(Dense(n_classes, activation='softmax'))

# Compile and train
ann_model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
ann_model.fit(X_train, y_train_ann, epochs=50, batch_size=16, verbose=0)
```

- Pemakaian SMOTE pada training sets

```
[ ] from imblearn.over_sampling import SMOTE

    # Check minimum samples in any class in y_train
    min_samples = y_train.value_counts().min()
    print(f"Minimum samples in any class in y_train: {min_samples}")

    # Apply SMOTE to training data only
    # Set k_neighbors to be less than or equal to the minimum number of samples in any class minus 1.
    # If min_samples is 5, k_neighbors should be at most 4.
    smote = SMOTE(random_state=42, k_neighbors=min_samples - 1) # Ensure k_neighbors < min_samples
    X_train_smote, y_train_smote = smote.fit_resample(X_train, y_train)

    # Check new class distribution
    print("Class distribution after SMOTE:\n", pd.Series(y_train_smote).value_counts().sort_index())
```

```
Minimum samples in any class in y_train: 5
Class distribution after SMOTE:
 quality
3    386
4    386
5    386
6    386
7    386
8    386
Name: count, dtype: int64
```

```
# SVM with SMOTE
print("\n===== SVM Classifier (with SMOTE) =====")
svm_model_smote = SVC(kernel='rbf', C=1, gamma='scale')
svm_model_smote.fit(X_train_smote, y_train_smote)
```

```
# KNN with SMOTE
print("\n===== KNN Classifier (with SMOTE) =====")
knn_model_smote = KNeighborsClassifier(n_neighbors=5)
knn_model_smote.fit(X_train_smote, y_train_smote)
```

```
# ANN with SMOTE
print("\n===== ANN Classifier (with SMOTE) =====")

# One-hot encode new SMOTE targets
y_train_smote_ann = to_categorical(y_train_smote)
y_test_ann = to_categorical(y_test)

# Build model
ann_model_smote = Sequential()
ann_model_smote.add(Dense(64, activation='relu', input_shape=(X_train_smote.shape[1],)))
ann_model_smote.add(Dense(32, activation='relu'))
ann_model_smote.add(Dense(y_train_smote_ann.shape[1], activation='softmax'))

# Compile and train
ann_model_smote.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
ann_model_smote.fit(X_train_smote, y_train_smote_ann, epochs=50, batch_size=16, verbose=0)
```

4. Evaluasi

Hasil (tanpa SMOTE):

```
y_pred_svm = svm_model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred_svm))
print("\nClassification Report:\n", classification_report(y_test, y_pred_svm))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_svm))
```

```
===== SVM Classifier =====
Accuracy: 0.6637554585152838

Classification Report:
              precision    recall  f1-score   support

           3       0.00      0.00      0.00         1
           4       0.00      0.00      0.00         7
           5       0.73      0.79      0.76        97
           6       0.61      0.72      0.66        92
           7       0.64      0.31      0.42        29
           8       0.00      0.00      0.00         3

    accuracy                           0.66       229
   macro avg       0.33      0.30      0.31       229
weighted avg       0.63      0.66      0.64       229

Confusion Matrix:
 [[ 0  0  1  0  0  0]
 [ 0  0  4  3  0  0]
 [ 0  0 77 20  0  0]
 [ 0  0 22 66  4  0]
 [ 0  0  2 18  9  0]
 [ 0  0  0  2  1  0]]
```

```
y_pred_knn = knn_model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred_knn))
print("\nClassification Report:\n", classification_report(y_test, y_pred_knn))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_knn))
```

```
===== KNN Classifier =====
Accuracy: 0.5676855895196506

Classification Report:
              precision    recall  f1-score   support

           3       0.00      0.00      0.00         1
           4       0.00      0.00      0.00         7
           5       0.67      0.66      0.66        97
           6       0.53      0.61      0.57        92
           7       0.40      0.34      0.37        29
           8       0.00      0.00      0.00         3

    accuracy                           0.57       229
   macro avg       0.27      0.27      0.27       229
weighted avg       0.55      0.57      0.56       229

Confusion Matrix:
 [[ 0  0  1  0  0  0]
 [ 0  0  2  5  0  0]
 [ 0  1 64 28  4  0]
 [ 0  1 25 56 10  0]
 [ 0  0  3 16 10  0]
 [ 0  0  1  1  1  0]]
```

```
y_pred_ann = ann_model.predict(X_test)
y_pred_ann_classes = np.argmax(y_pred_ann, axis=1)

print("Accuracy:", accuracy_score(y_test, y_pred_ann_classes))
print("\nClassification Report:\n", classification_report(y_test, y_pred_ann_classes))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_ann_classes))
```

```
===== ANN Classifier =====
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step
Accuracy: 0.6550218340611353

Classification Report:
              precision    recall  f1-score   support

           3       0.00      0.00      0.00         1
           4       0.00      0.00      0.00         7
           5       0.72      0.76      0.74        97
           6       0.62      0.70      0.65        92
           7       0.57      0.41      0.48        29
           8       0.00      0.00      0.00         3

    accuracy                           0.66       229
   macro avg       0.32      0.31      0.31       229
weighted avg       0.62      0.66      0.64       229

Confusion Matrix:
 [[ 0  0  1  0  0  0]
 [ 0  0  5  2  0  0]
 [ 0  0 74 21  2  0]
 [ 0  0 22 64  6  0]
 [ 0  0  1 15 12  1]
 [ 0  0  0  2  1  0]]
```

Model	Accuracy	Macro F1-Score
SVM	66.4%	~0.31
KNN	56.8%	~0.27
ANN	65.5%	~0.31

Hasil (dengan SMOTE):

```
[ ]
y_pred_svm_smote = svm_model_smote.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred_svm_smote))
print("\nClassification Report:\n", classification_report(y_test, y_pred_svm_smote))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_svm_smote))
```

```
===== SVM Classifier (with SMOTE) =====
Accuracy: 0.5633187772925764

Classification Report:
              precision    recall  f1-score   support

           3       0.00      0.00      0.00         1
           4       0.10      0.29      0.15         7
           5       0.73      0.68      0.70        97
           6       0.68      0.53      0.60        92
           7       0.41      0.38      0.39        29
           8       0.07      0.33      0.11         3

    accuracy                           0.56       229
   macro avg       0.33      0.37      0.33       229
weighted avg       0.64      0.56      0.59       229

Confusion Matrix:
 [[ 0  0  1  0  0  0]
 [ 0  2  3  2  0  0]
 [ 4  8 66 16  2  1]
 [ 0  7 19 49 12  5]
 [ 0  3  2  5 11  8]
 [ 0  0  0  0  2  1]]
```

```
y_pred_knn_smote = knn_model_smote.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred_knn_smote))
print("\nClassification Report:\n", classification_report(y_test, y_pred_knn_smote))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_knn_smote))
```

```
===== KNN Classifier (with SMOTE) =====
Accuracy: 0.45414847161572053

Classification Report:
              precision    recall  f1-score   support

           3       0.00      0.00      0.00         1
           4       0.03      0.14      0.06         7
           5       0.69      0.54      0.60        97
           6       0.53      0.40      0.46        92
           7       0.33      0.48      0.39        29
           8       0.00      0.00      0.00         3

    accuracy                           0.45       229
   macro avg       0.26      0.26      0.25       229
weighted avg       0.55      0.45      0.49       229

Confusion Matrix:
 [[ 0  0  1  0  0  0]
 [ 0  1  2  4  0  0]
 [ 1 15 52 23  4  2]
 [ 0 10 17 37 23  5]
 [ 0  3  2  6 14  4]
 [ 0  0  1  0  2  0]]
```

```
[ ] y_pred_ann_smote = ann_model_smote.predict(X_test)
    y_pred_ann_smote_classes = np.argmax(y_pred_ann_smote, axis=1)

    print("Accuracy:", accuracy_score(y_test, y_pred_ann_smote_classes))
    print("\nClassification Report:\n", classification_report(y_test, y_pred_ann_smote_classes))
    print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_ann_smote_classes))
```

```
===== ANN Classifier (with SMOTE) =====
8/8 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
Accuracy: 0.611353711790393

Classification Report:
              precision    recall  f1-score   support

           3       0.00      0.00      0.00         1
           4       0.00      0.00      0.00         7
           5       0.71      0.74      0.72        97
           6       0.65      0.55      0.60        92
           7       0.45      0.59      0.51        29
           8       0.00      0.00      0.00         3

    accuracy                           0.61       229
   macro avg       0.30      0.31      0.30       229
weighted avg       0.62      0.61      0.61       229

Confusion Matrix:
 [[ 0  0  1  0  0  0]
 [ 0  0  4  2  1  0]
 [ 2  2 72 17  4  0]
 [ 0  3 21 51 16  1]
 [ 0  0  3  7 17  2]
 [ 0  0  1  2  0  0]]
```

Model	Accuracy	Macro F1-Score
SVM (With SMOTE)	56.3%	~0.33
KNN (With SMOTE)	45.4%	~0.25
ANN (With SMOTE)	61.1%	~0.30

Analisis:
- SMOTE meningkatkan recall pada kelas minoritas, tetapi menurunkan akurasi secara keseluruhan
- ANN menunjukkan performa paling seimbang di antara model lain
- Dari 3 model tersebut selalu susah di prediksi pada kelas 3,4 dan 8 karena terbatasnya / sedikit jumlah data di test set meski sudah di SMOTE

2. Silahkan gunakan dataset public yang bisa digunakan untuk clustering. Adapun dataset tersebut:
   - Jumlah fiturnya minimal 7
   - Jumlah total datanya minimal 1000.

   Pertanyaan:

   a. Tunjukkan data linknya dan lakukan EDA (Exploratory Data Analysis) dari dataset yang digunakan **(LO3) (5 point)**
   Dataset: Most Streamed Spotify Songs 2024
   Link Dataset: https://www.kaggle.com/datasets/nelgiriyewithana/most-streamed-spotify-songs-2024
   Link ipynb :
   https://colab.research.google.com/drive/1YqQsx4aiXtVYLGO9505w_MtB_Xmn5-_1?usp=sharing

   Jumlah Fitur Terpilih: 7 fitur numerik
   - Track Score
   - Spotify Streams
   - Spotify Popularity
   - Apple Music Playlist Count
   - Deezer Playlist Count
   - Amazon Playlist Count
   - Explicit Track

	Track	Album Name	Artist	Release Date	ISRC	All Time Rank	Track Score	Spotify Streams	Spotify Playlist Count	Spotify Playlist Reach	...	SiriusXM Spins	Deezer Playlist Count	Deezer Playlist Reach	Amazon Playlist Count	Pandora Streams	Pandora Track Stations	Soundcloud Streams	Sha Cou
0	MILLION DOLLAR BABY	Million Dollar Baby - Single	Tommy Richman	4/26/2024	QM24S2402528	1	725.4	390,470,936	30,716	196,631,588	...	684	62.0	17,598,718	114.0	18,004,655	22,931	4,818,457	2,669
1	Not Like Us	Not Like Us	Kendrick Lamar	5/4/2024	USUG12400910	2	545.9	323,703,884	28,113	174,597,137	...	3	67.0	10,422,430	111.0	7,780,028	28,444	6,623,075	1,118
2	i like the way you kiss me	I like the way you kiss me	Artemas	3/19/2024	QZJ842400387	3	538.4	601,309,283	54,331	211,607,669	...	536	136.0	36,321,847	172.0	5,022,621	5,639	7,208,651	5,285
3	Flowers	Flowers - Single	Miley Cyrus	1/12/2023	USSM12209777	4	444.9	2,031,280,633	269,802	136,569,078	...	2,182	264.0	24,684,248	210.0	190,260,277	203,384	NaN	11,822
4	Houdini	Houdini	Eminem	5/31/2024	USUG12403398	5	423.3	107,034,922	7,223	151,469,874	...	1	82.0	17,660,624	105.0	4,493,884	7,006	207,179	457

5 rows × 29 columns

EDA Summary:

- Informasi umum data

```
Dataset shape: (4600, 29)

Dataset info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4600 entries, 0 to 4599
Data columns (total 29 columns):
 #   Column                      Non-Null Count  Dtype
---  ------                      --------------  -----
 0   Track                       4600 non-null   object
 1   Album Name                  4600 non-null   object
 2   Artist                      4595 non-null   object
 3   Release Date                4600 non-null   object
 4   ISRC                        4600 non-null   object
 5   All Time Rank               4600 non-null   object
 6   Track Score                 4600 non-null   float64
 7   Spotify Streams             4487 non-null   object
 8   Spotify Playlist Count      4530 non-null   object
 9   Spotify Playlist Reach      4528 non-null   object
 10  Spotify Popularity          3796 non-null   float64
 11  YouTube Views               4292 non-null   object
 12  YouTube Likes               4285 non-null   object
 13  TikTok Posts                3427 non-null   object
 14  TikTok Likes                3620 non-null   object
 15  TikTok Views                3619 non-null   object
 16  YouTube Playlist Reach      3591 non-null   object
 17  Apple Music Playlist Count  4039 non-null   float64
 18  AirPlay Spins               4102 non-null   object
 19  SiriusXM Spins              2477 non-null   object
 20  Deezer Playlist Count       3679 non-null   float64
 21  Deezer Playlist Reach       3672 non-null   object
 22  Amazon Playlist Count       3545 non-null   float64
 23  Pandora Streams             3494 non-null   object
 24  Pandora Track Stations      3332 non-null   object
 25  Soundcloud Streams          1267 non-null   object
 26  Shazam Counts               4023 non-null   object
 27  TIDAL Popularity            0 non-null      float64
 28  Explicit Track              4600 non-null   int64
dtypes: float64(6), int64(1), object(22)
memory usage: 1.0+ MB
```

- Check Missing value

```
# Check for missing values
missing_values = df.isnull().sum()
print("\nMissing values per column:")
print(missing_values[missing_values > 0])
```

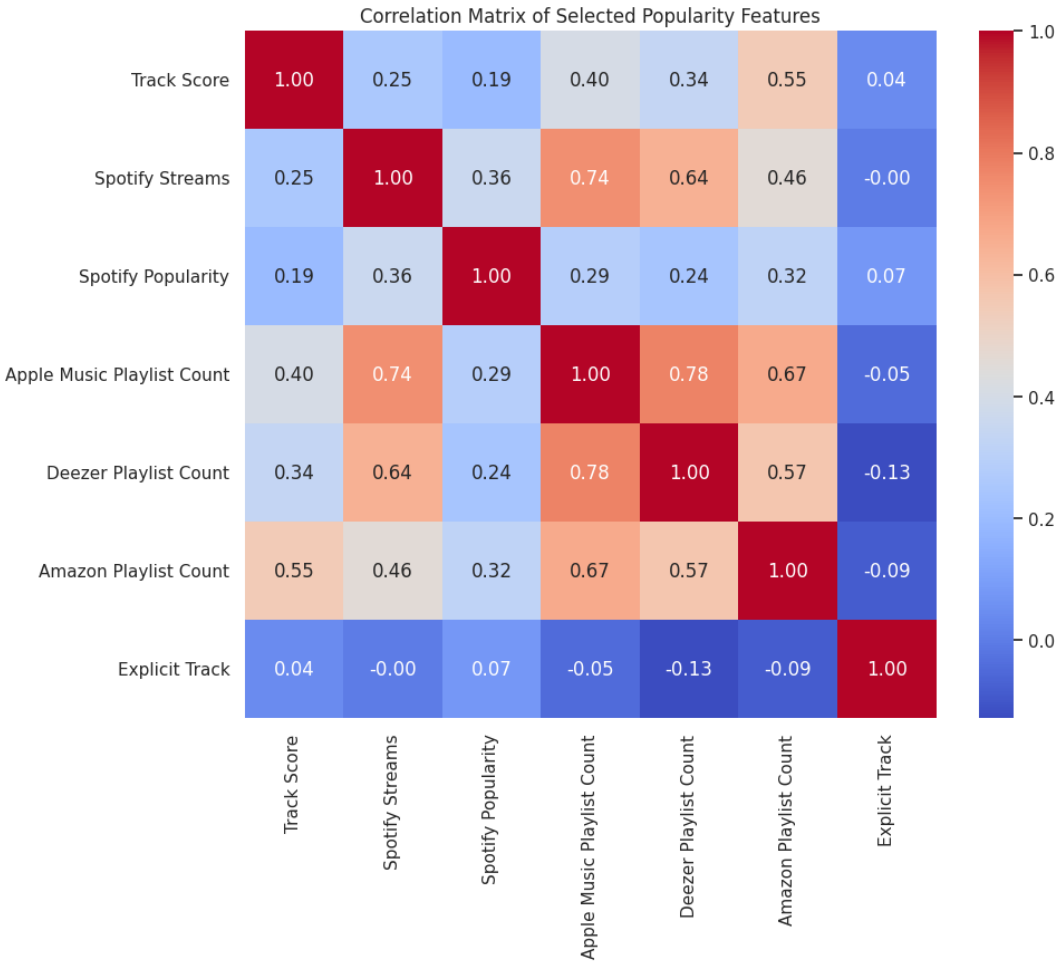
```
Missing values per column:
Artist                          5
Spotify Streams               113
Spotify Playlist Count         70
Spotify Playlist Reach         72
Spotify Popularity            804
YouTube Views                 308
YouTube Likes                 315
TikTok Posts                 1173
TikTok Likes                  980
TikTok Views                  981
YouTube Playlist Reach       1009
Apple Music Playlist Count    561
AirPlay Spins                 498
SiriusXM Spins               2123
Deezer Playlist Count         921
Deezer Playlist Reach         928
Amazon Playlist Count        1055
Pandora Streams              1106
Pandora Track Stations       1268
Soundcloud Streams           3333
Shazam Counts                 577
TIDAL Popularity             4600
dtype: int64
```

- Descriptive Statistics

	Track Score	Spotify Streams	Spotify Popularity	Apple Music Playlist Count	Deezer Playlist Count	Amazon Playlist Count	Explicit Track
count	4600.000000	4.487000e+03	3796.000000	4039.00000	3679.000000	3545.000000	4600.000000
mean	41.844043	4.473873e+08	63.501581	54.60312	32.310954	25.348942	0.358913
std	38.543766	5.384439e+08	16.186438	71.61227	54.274538	25.989826	0.479734
min	19.400000	1.071000e+03	1.000000	1.00000	1.000000	1.000000	0.000000
25%	23.300000	7.038630e+07	61.000000	10.00000	5.000000	8.000000	0.000000
50%	29.900000	2.398507e+08	67.000000	28.00000	15.000000	17.000000	0.000000
75%	44.425000	6.283638e+08	73.000000	70.00000	37.000000	34.000000	1.000000
max	725.400000	4.281469e+09	96.000000	859.00000	632.000000	210.000000	1.000000

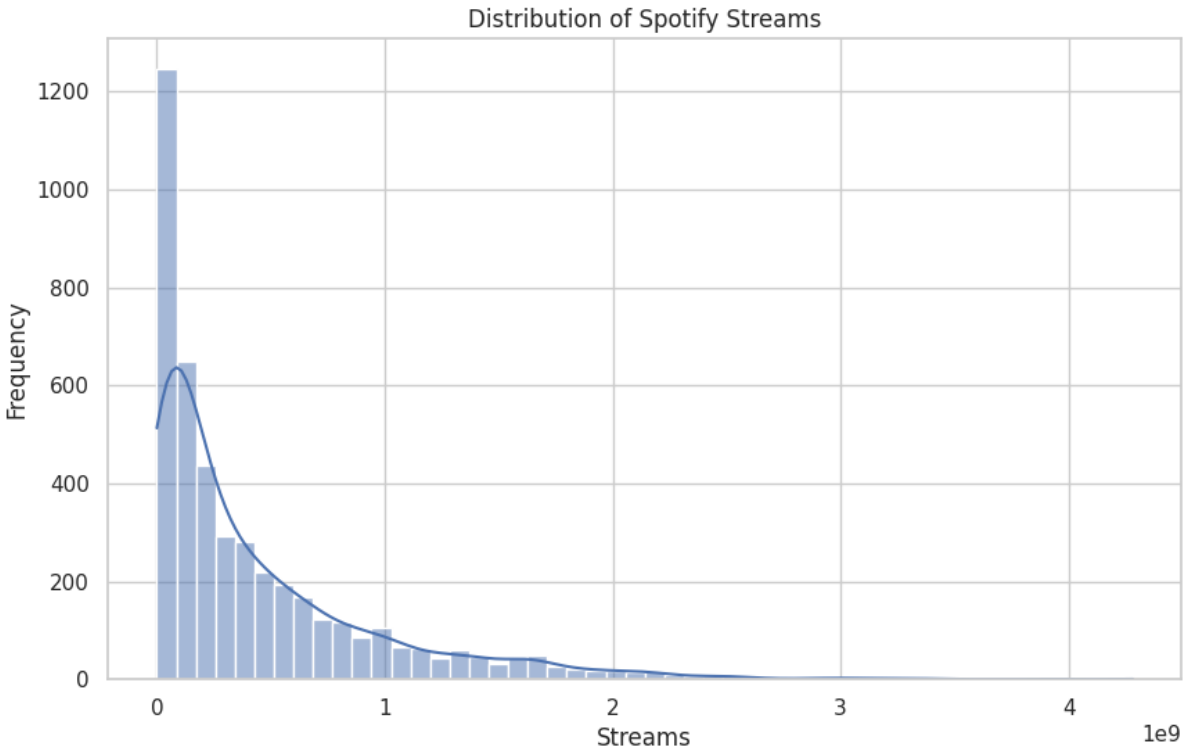
Track Score berkemunginan outlier

- Correlation Matrix



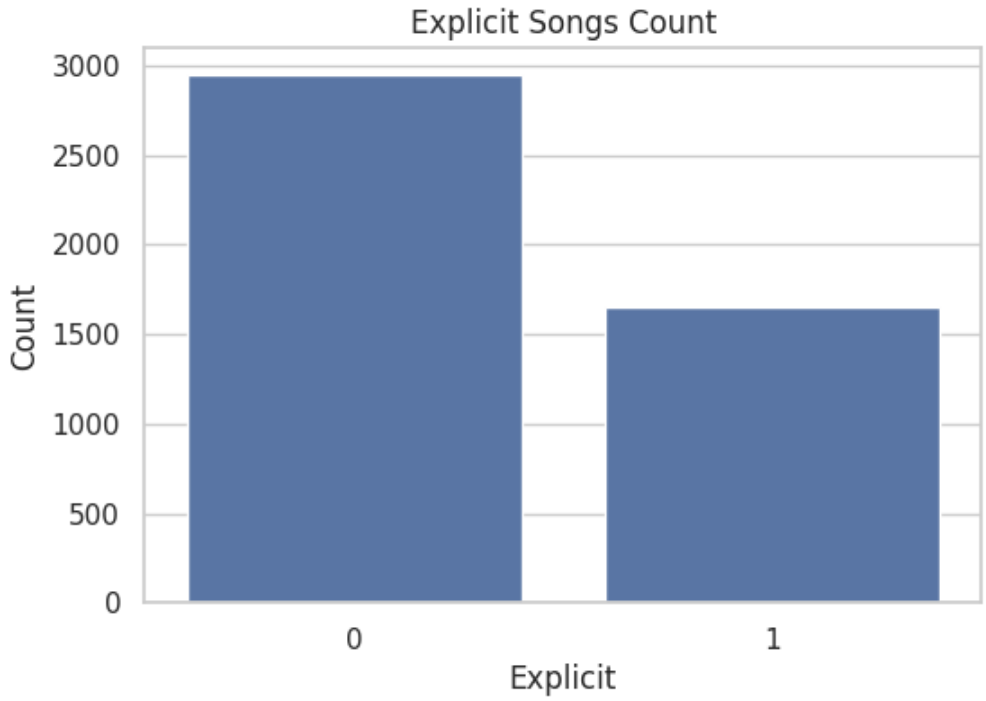
Terdapat beberapa nilai yang tinggi korelasinya antar platform sehingga dibutuhkan PCA untuk menurunkan redudansi

- Spotify Streams Distribution



Sangat Right Skewed yang mengindikasikan jumlah lagu hanya sedikit yang terkenal, dimana mayoritas realtif memiliki nilai stream yang kecil sehingga dibutuhkan standarisasi

- Explicit Content Distribution



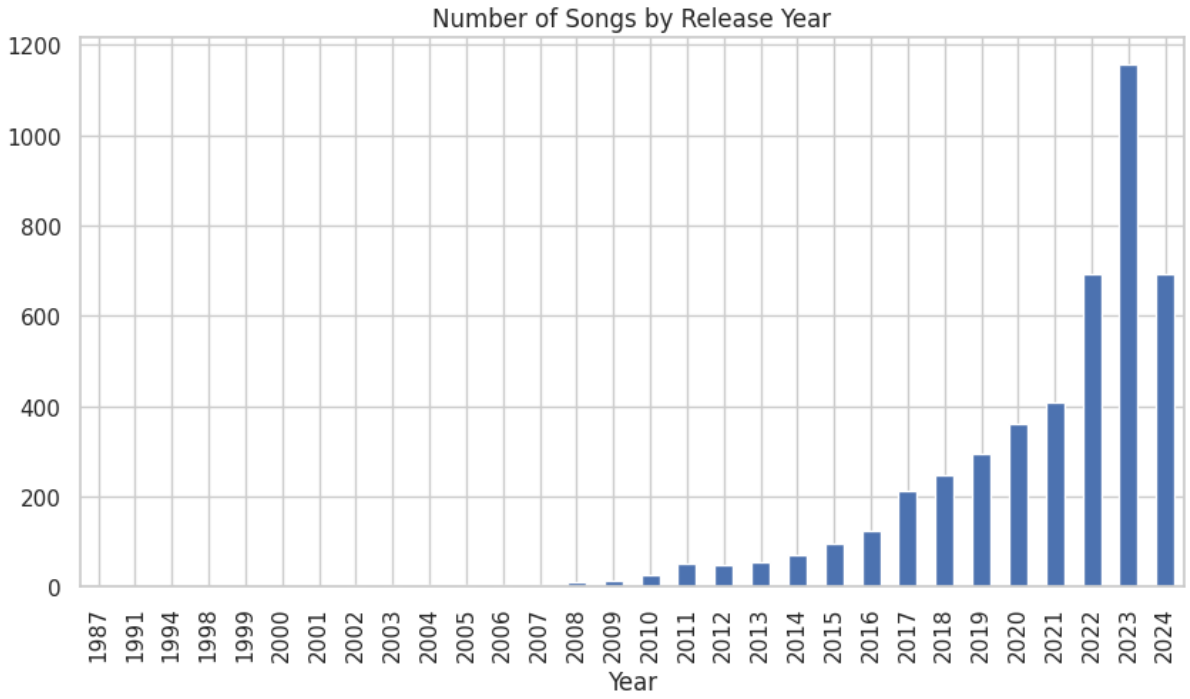
Hanya sekitar 36% lagu memiliki label explicit

- Top 10 Artist

Artist	count
Taylor Swift	63
Drake	63
Bad Bunny	60
KAROL G	32
The Weeknd	31
Travis Scott	30
Billie Eilish	27
Ariana Grande	26
Future	23
Post Malone	22

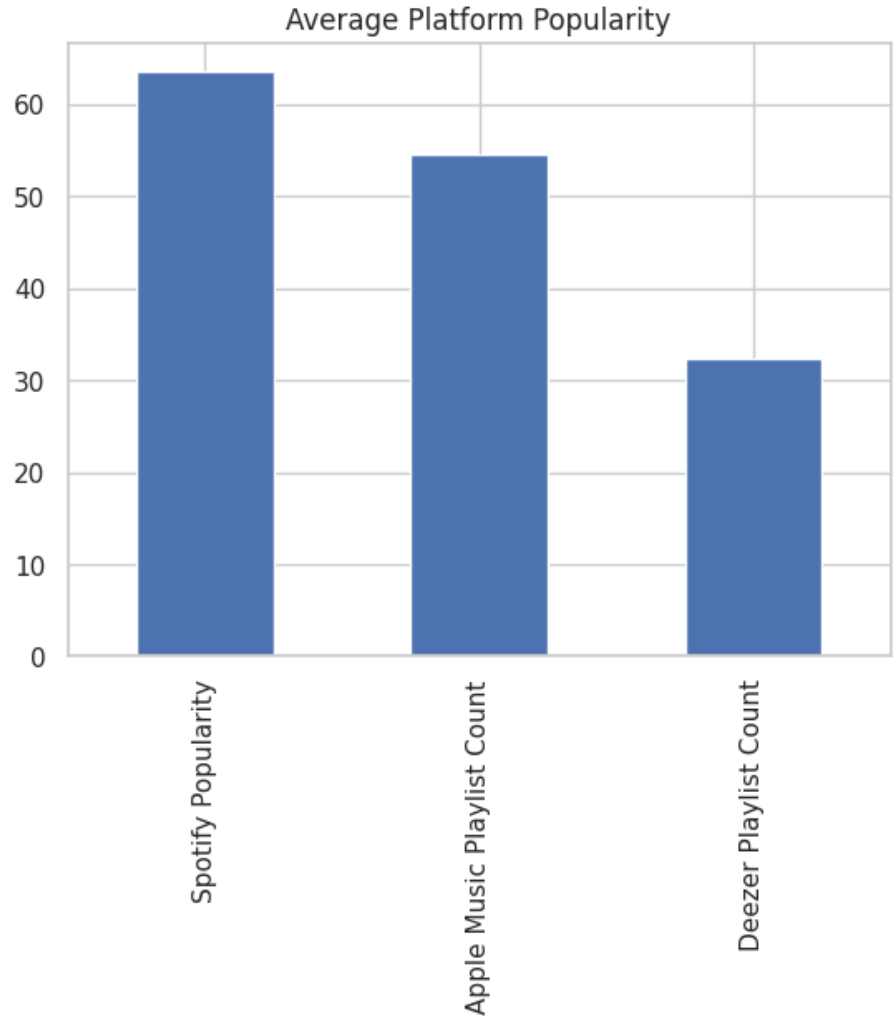
Beberapa artis paling dominan: Drake, Taylor Swift, Bad Bunny. Artist tersebut mungkin saja terhitung tinggi karena duplikasi gu atau penampilan.

- Release Year



Mayoritas lagu berasal dari tahun 2022–2024

- Platform Popularity



- Spotify mendominasi dalam hal popularitas rata-rata

b. Lakukan klasifikasi dengan K-Means. Tunjukan hasil performancenya (multi class) dan lakukan analisis **(LO1, LO2, LO4) (20 point)**

Tahapan kerja:

1. Setup and Library Import

```
# Import necessary libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.decomposition import PCA
import warnings


sns.set(style="whitegrid")
warnings.filterwarnings("ignore")
```

2. **Read and preview dataset:**

```
[2]  # Load dataset
     df = pd.read_csv("Most Streamed Spotify Songs 2024.csv", encoding="latin1")

     # Lihat 5 data teratas
     df.head()
```

3. **EDA Terdiri dari (Sudah dibahas di A)**
   - Informasi Data
   - Check missing value
   - Evaluasi Statistik
   - Correlasi Fitur
   - Distribusi Stream Spotify
   - Distribusi Konten Explisit
   - Top 10 Artis terbanyak
   - Distribusi Tahun Rilis
   - Perbandingkan popularitas platform
4. **Data Preprocessing**
   - Pengambilan 7 fitur penting untuk clustering
   - Hapus data yang missing value
   - Standarisasi data

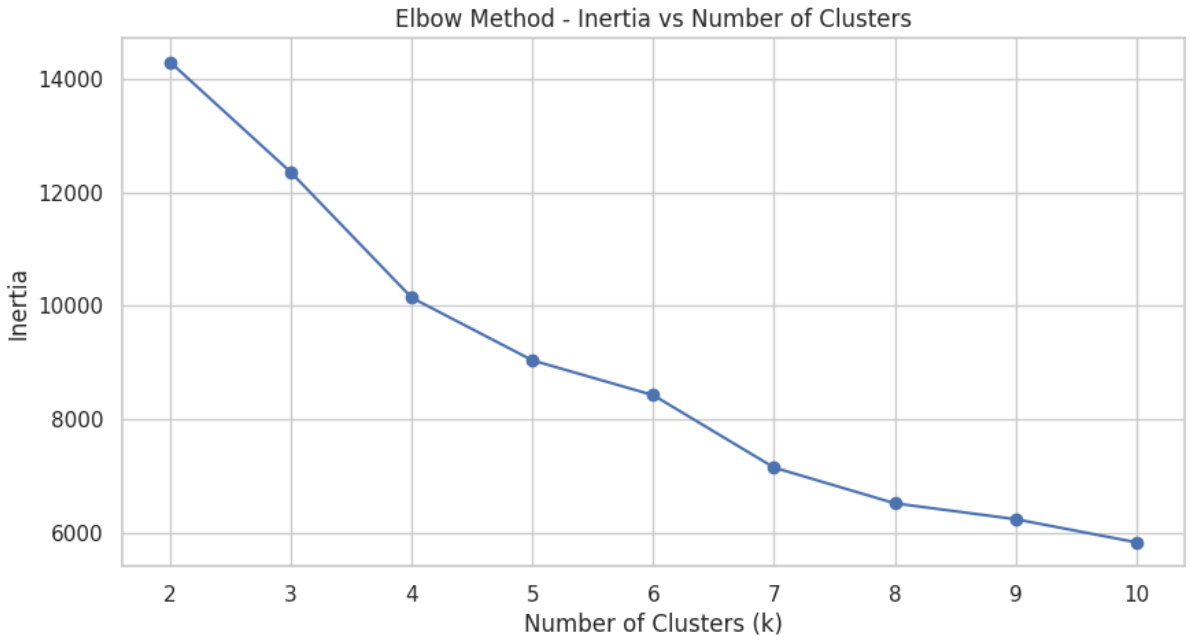
```
# Select relevant numerical features for clustering
selected_features = [
    'Track Score',
    'Spotify Popularity',
    'Apple Music Playlist Count',
    'Deezer Playlist Count',
    'Amazon Playlist Count',
    'Explicit Track',
    'Spotify Streams'
]

# Drop rows with missing values in selected features
clustering_data = df[selected_features].dropna()

# Convert 'Spotify Streams' to numeric by removing commas
clustering_data['Spotify Streams'] = clustering_data['Spotify Streams'].astype(str).str.replace(',', '', regex=False).astype(float
```

```
[13] # Check shape before and after dropna
     print("Original rows:", df.shape[0])
     print("Rows after dropping missing values:", clustering_data.shape[0])

     Original rows: 4600
     Rows after dropping missing values: 2941
```

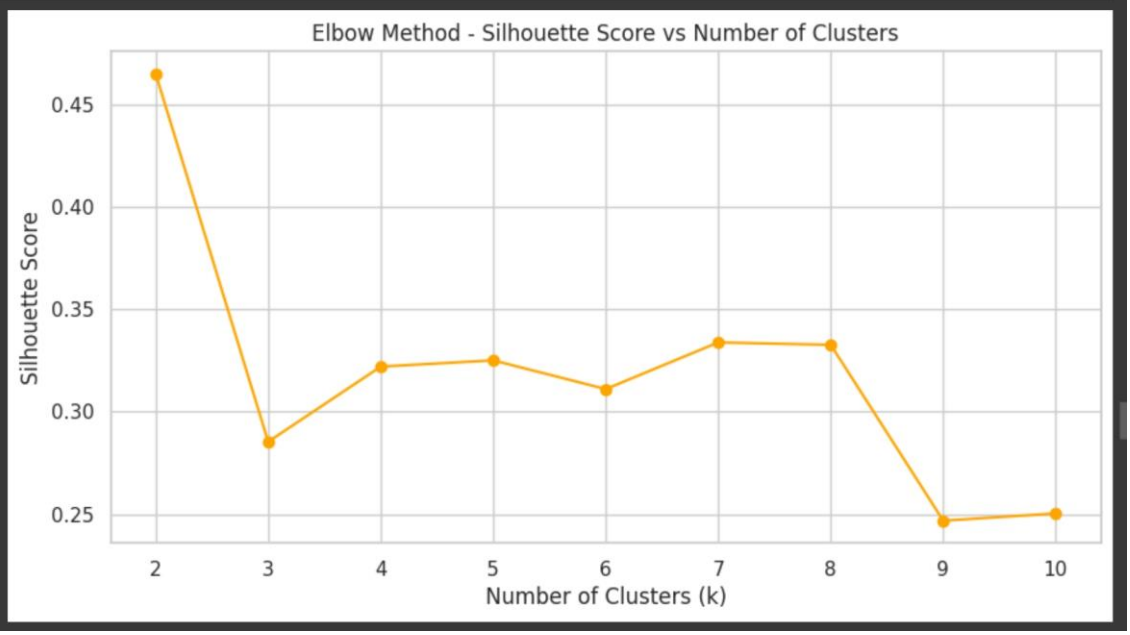
```
[14] # Standardize the data
     scaler = StandardScaler()
     scaled_data = scaler.fit_transform(clustering_data)
```

5. **K-Means Clustering**
   - Mencari nilai k terbaik untuk kasus ini dengan Inertia dan juga Silhoutte melalui Elbow Method
   Dimana mencari k yang terkecil tapi dengan nilai Inertia terendah + nilai Silhoutte terbesar

```python
# Determine optimal number of clusters with Elbow Method and Silhouette Score
inertia = []
silhouette_scores = []
K_range = range(2, 11)

for k in K_range:
    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(scaled_data)
    inertia.append(kmeans.inertia_)
    score = silhouette_score(scaled_data, kmeans.labels_)
    silhouette_scores.append(score)

# Plot Elbow Curve
plt.figure(figsize=(10, 5))
plt.plot(K_range, inertia, marker='o')
plt.title('Elbow Method - Inertia vs Number of Clusters')
plt.xlabel('Number of Clusters (k)')
plt.ylabel('Inertia')
plt.grid(True)
plt.show()
```



```
[16] # Plot Silhouette Scores
     plt.figure(figsize=(10, 5))
     plt.plot(K_range, silhouette_scores, marker='o', color='orange')
     plt.title('Elbow Method - Silhouette Score vs Number of Clusters')
     plt.xlabel('Number of Clusters (k)')
     plt.ylabel('Silhouette Score')
     plt.grid(True)
     plt.show()
```

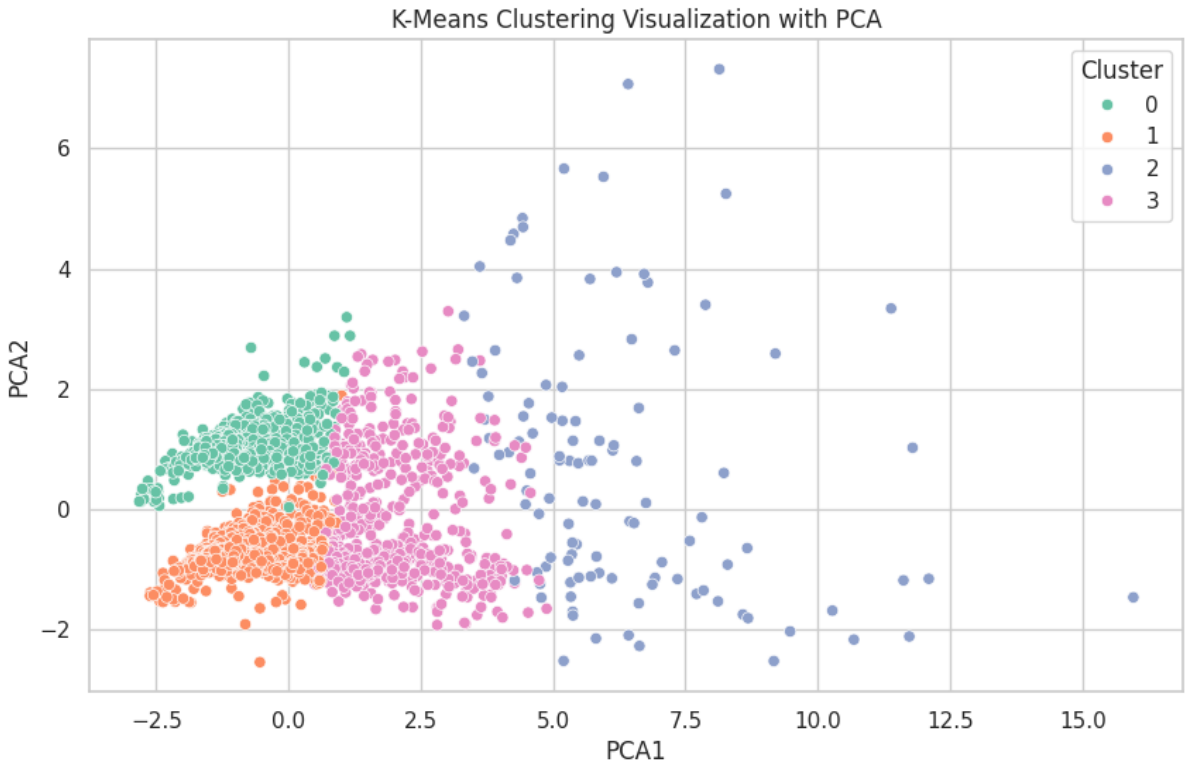


Dari gabungan 2 nilai tersebut lebih memungkinkan dimana nilai k = 4

6. **Cluster Visualization and Interpretation**

   Setelah mendapatkan nilai cluster k = 4 maka mulai finalisasi clutering dengan K-Means yang sudah melalui tahapan Reduksi dimensi dengan PCA

```
[17] # Apply KMeans with chosen number of clusters (adjust based on elbow/silhouette)
     k = 4
     kmeans_final = KMeans(n_clusters=k, random_state=42)
     clusters = kmeans_final.fit_predict(scaled_data)

     # Add cluster labels back to dataframe
     clustering_data['Cluster'] = clusters

     # Use PCA to reduce to 2D for visualization
     pca = PCA(n_components=2)
     pca_result = pca.fit_transform(scaled_data)
     clustering_data['PCA1'] = pca_result[:, 0]
     clustering_data['PCA2'] = pca_result[:, 1]

     # Visualize clusters
     plt.figure(figsize=(10, 6))
     sns.scatterplot(x='PCA1', y='PCA2', hue='Cluster', data=clustering_data, palette='Set2')
     plt.title('K-Means Clustering Visualization with PCA')
     plt.show()
```




```
[19] # Menampilkan jumlah lagu per cluster
     print("Cluster counts:\n", clustering_data['Cluster'].value_counts())
```

```
Cluster counts:
 Cluster
1    1335
0     940
3     556
2     110
Name: count, dtype: int64
```

7. Conclusion

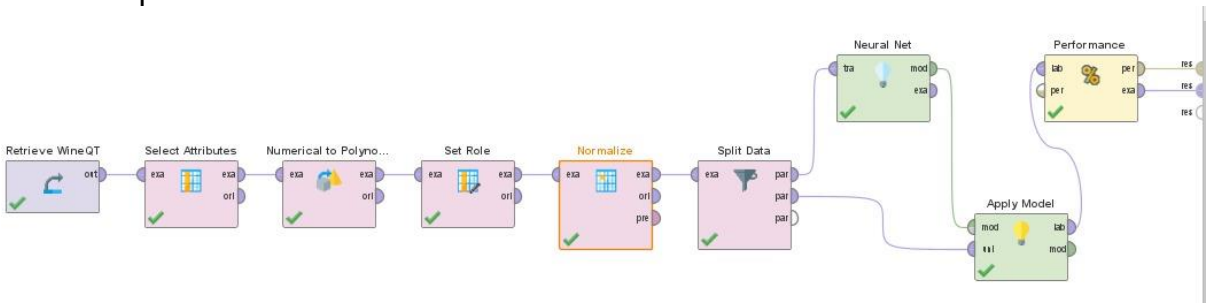
Pemilihan K: k = 4 (hasil dari Elbow dan Silhouette method)
Visualisasi: PCA menunjukkan pemisahan klaster yang cukup jelas
Interpretasi Klaster:
- Cluster 0: Lagu-lagu rata-rata, moderate di semua metrik
- Cluster 1: Lagu-lagu terpopuler, high on every metric
- Cluster 2: Lagu emerging, popular di playlist tapi rendah pada popularitas
- Cluster 3: Lagu-lagu eksplisit dan edgy

3. Dengan menggunakan Rapid Miner tunjukkan hasil salah satu algoritma klasifikasi no 1 dan clustering pada no 2 **(LO2) (10 point)**

Klasifikasi

Model: Artificial Neural Network

Tahapan

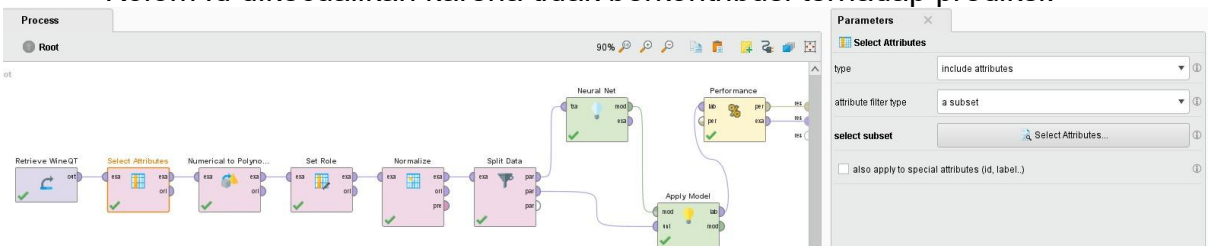


- **Read CSV**

Dataset dimuat menggunakan operator Read CSV. File yang digunakan adalah dataset WineQT.csv.

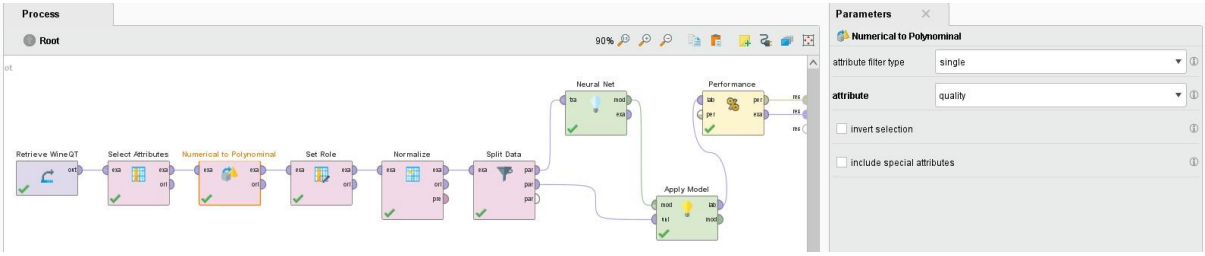
- **Select Attributes**

Kolom Id dikecualikan karena tidak berkontribusi terhadap prediksi.



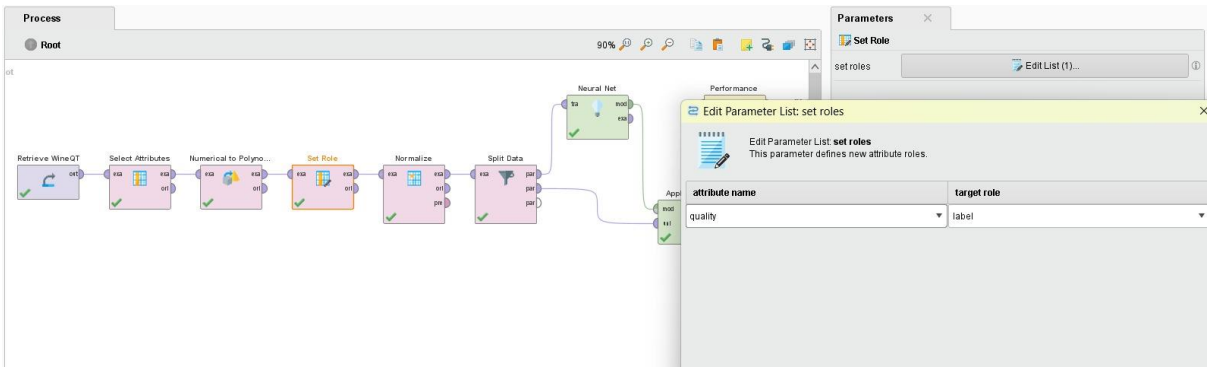
- **Numerical to Polynomial**

Karena operator Performance (Classification) membutuhkan label nominal, maka quality diubah dari numerik menjadi nominal (polynomial).



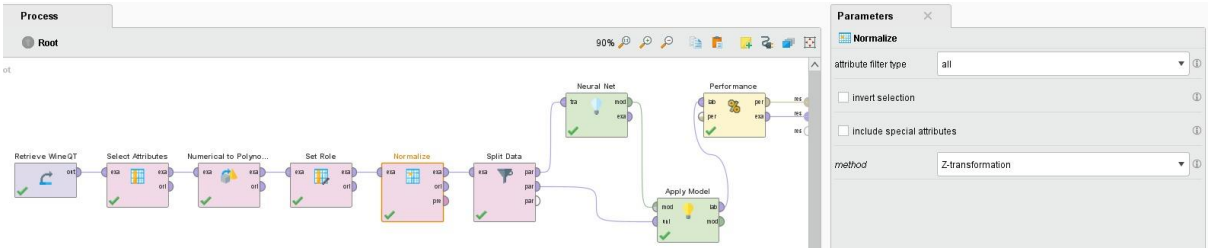
• **Set Role**

Kolom quality diatur sebagai label menggunakan operator Set Role.



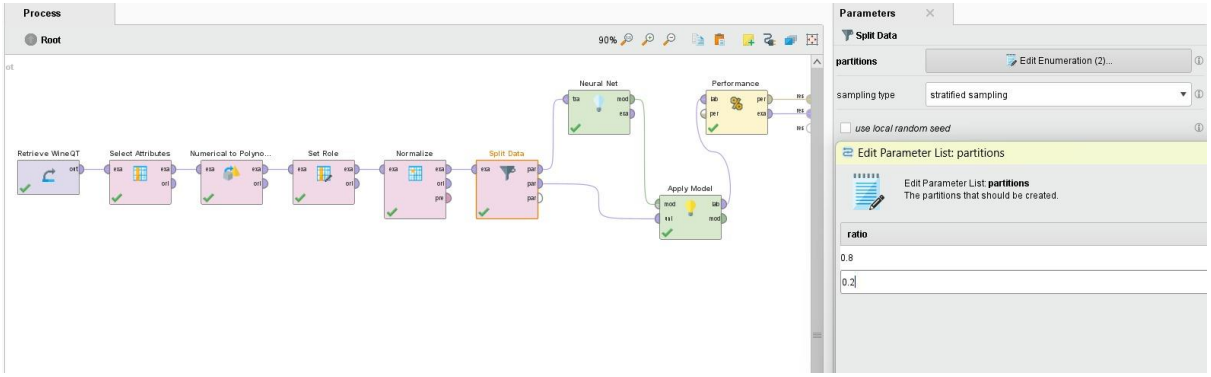
• **Z-Transformation (Normalize)**

Semua atribut numerik dinormalisasi menggunakan Z-Transformation agar model ANN tidak dipengaruhi oleh skala fitur.

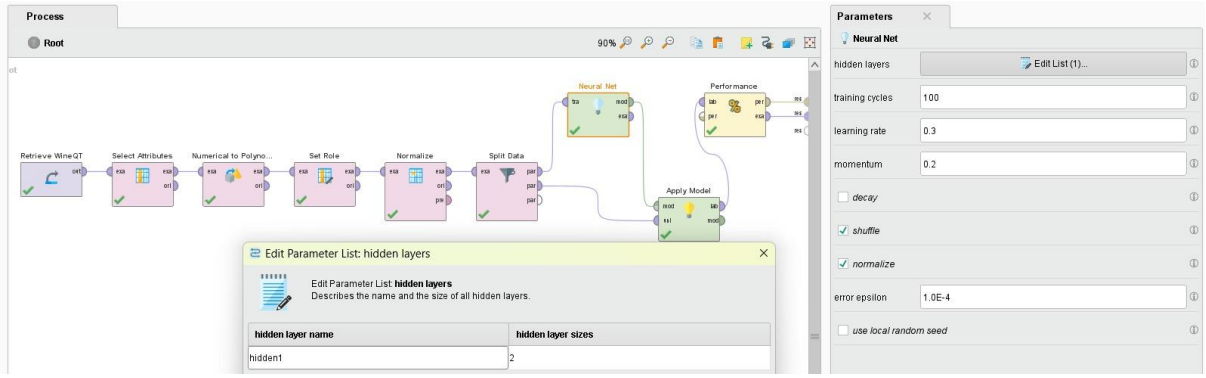


• **Split Data (Stratified)**

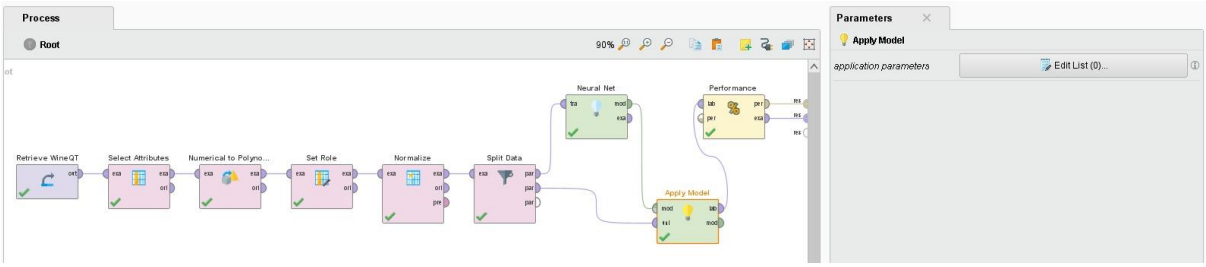
Data dibagi menjadi 80% training dan 20% testing dengan menjaga proporsi kelas (stratified sampling).



• **Train Model (Neural Net)**
Model ANN dilatih menggunakan operator Neural Net. Parameter seperti jumlah hidden layers dan epoch disesuaikan default RapidMiner.



• **Apply Model**
Model diterapkan pada data testing yang sudah dipisah sebelumnya.

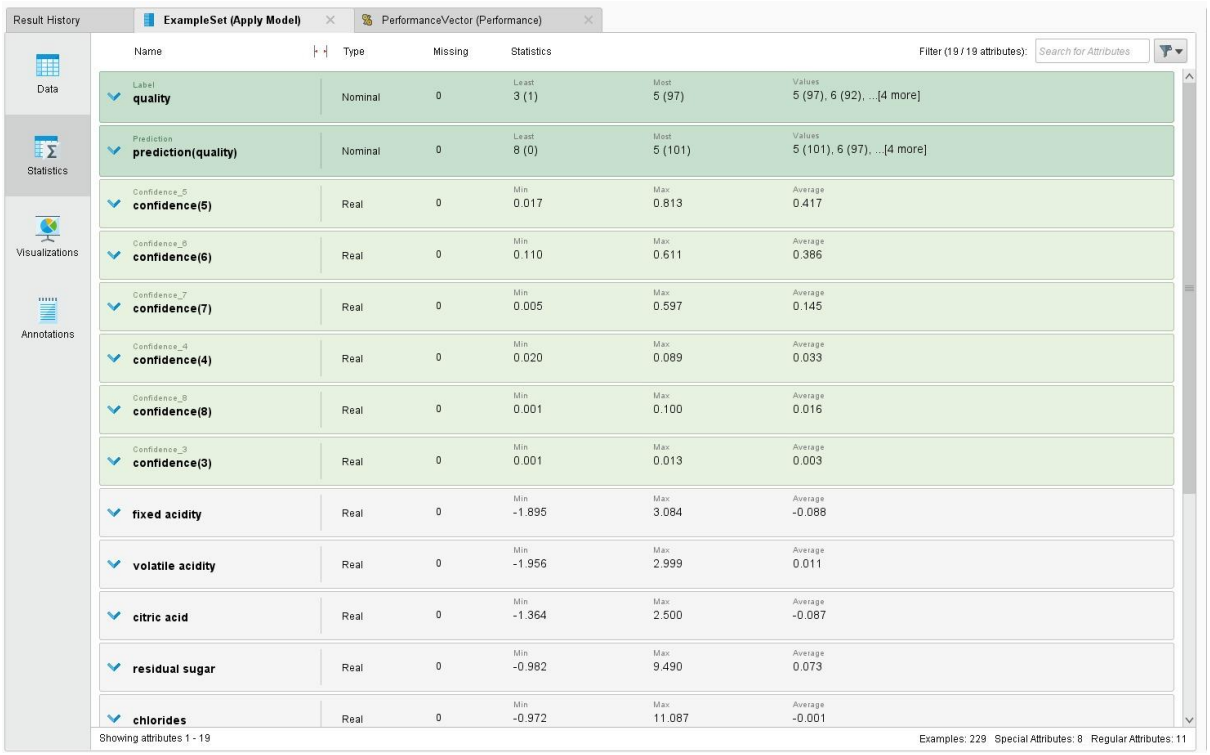


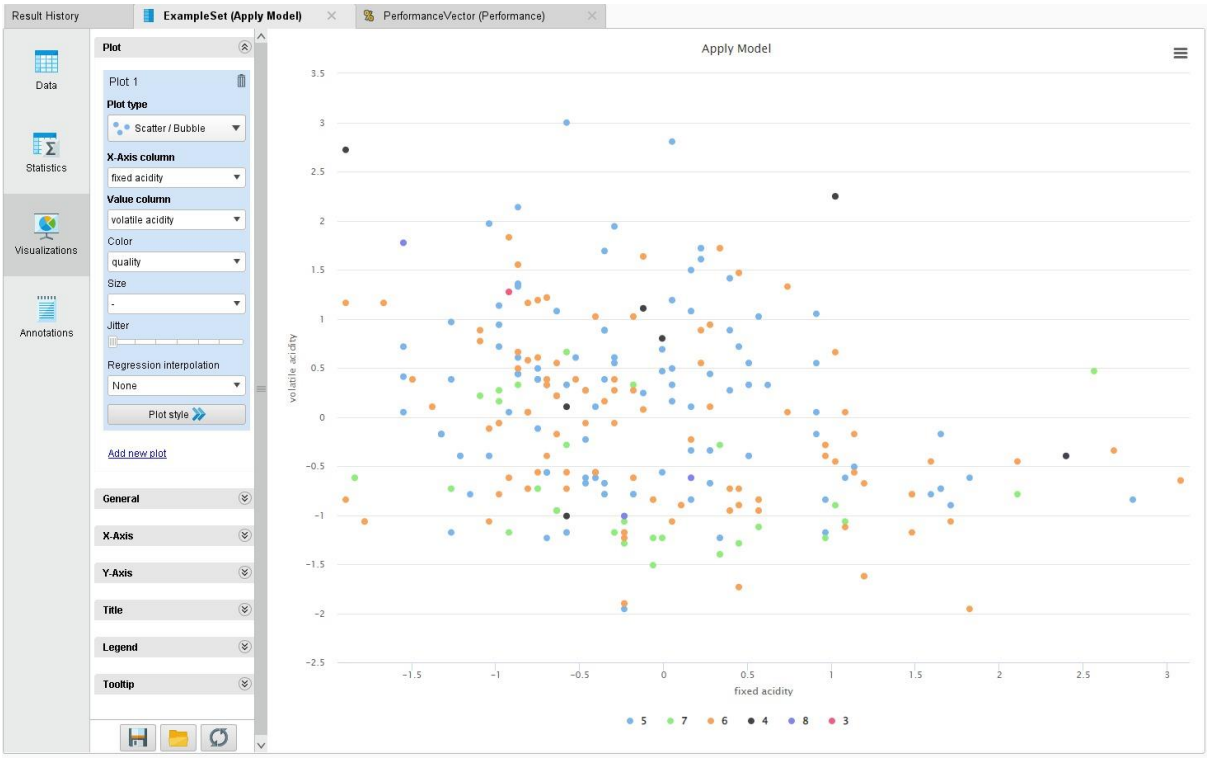
• **Performance (Classification)**
Evaluasi dilakukan menggunakan operator Performance (Classification) dengan metrik akurasi dan per kelas precision & recall.

Hasil Evaluasi ANN di RapidMiner:
• **Akurasi**: 55.02%
• **Precision tertinggi** ada pada kelas 5 (66.34%) dan kelas 6 (51.55%)
• **Recall tertinggi** juga di kelas 5 dan 6
• Kelas 3, 4, dan 8 tidak terprediksi dengan baik (precision dan recall = 0)

accuracy: 55.02%

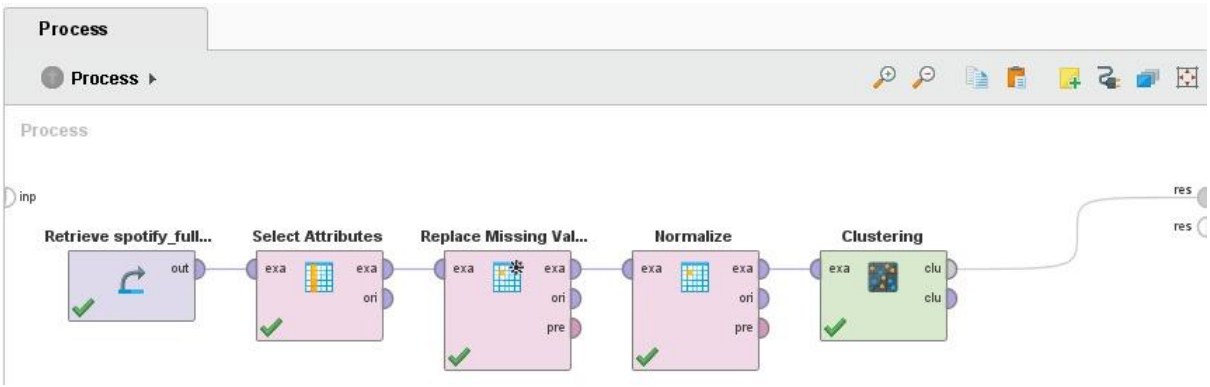
	true 5	true 6	true 7	true 4	true 8	true 3	class precision
pred. 5	67	25	3	5	0	1	66.34%
pred. 6	26	50	17	2	2	0	51.55%
pred. 7	4	17	9	0	1	0	29.03%
pred. 4	0	0	0	0	0	0	0.00%
pred. 8	0	0	0	0	0	0	0.00%
pred. 3	0	0	0	0	0	0	0.00%
class recall	69.07%	54.35%	31.03%	0.00%	0.00%	0.00%	





• Ketidakseimbangan data menyebabkan kelas minoritas tidak terdeteksi
• Model ANN cocok diterapkan karena menunjukkan kestabilan pada fitur yang telah dinormalisasi

Clustering

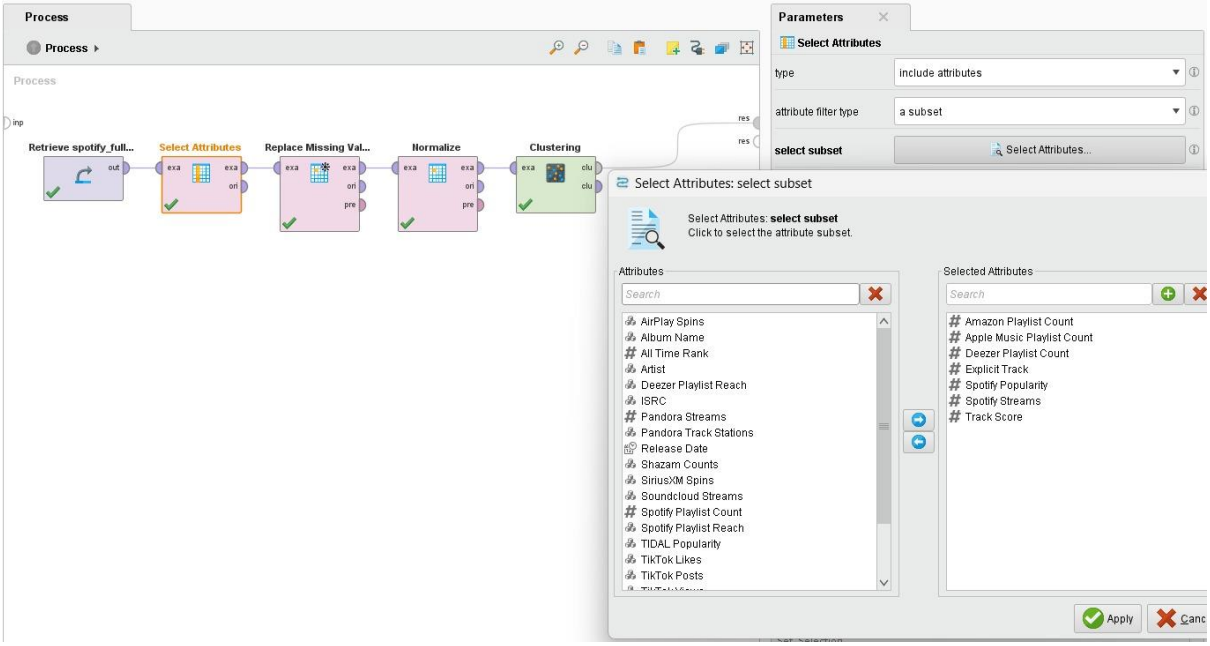


Langkah-langkah Clustering K-Means di RapidMiner

1. Import Dataset

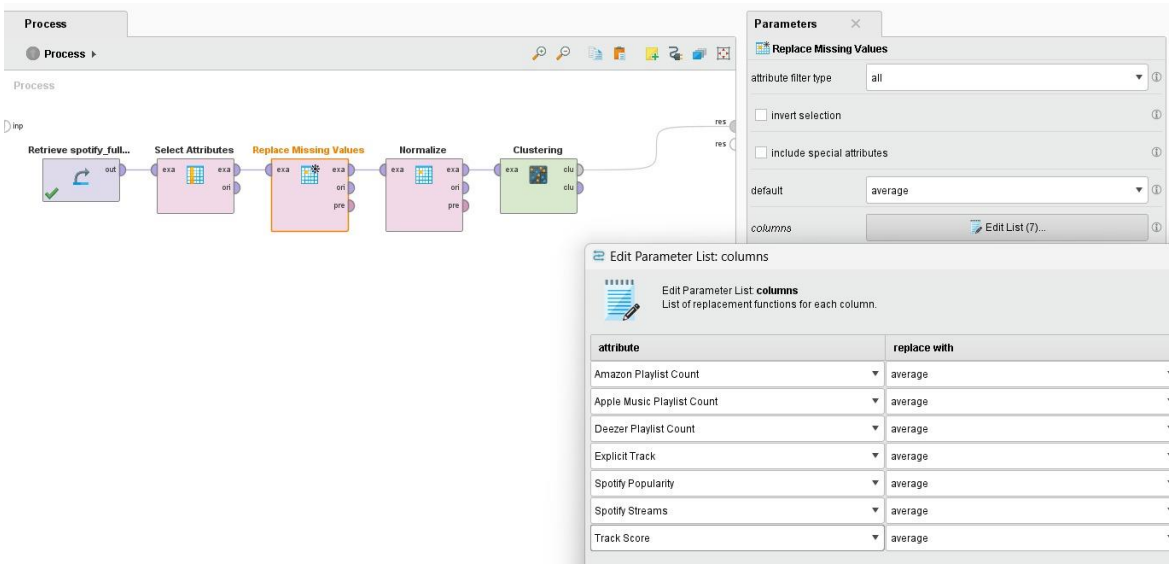
2. Select Attributes

• Memilih hanya 7 fitur numerik yang relevan.



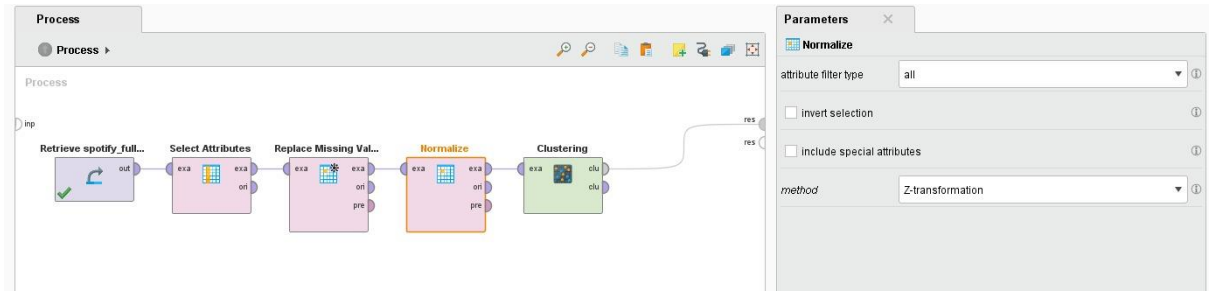
3. Replace Missing Values

• Menggunakan nilai rata-rata (average).



4. Normalisasi Data

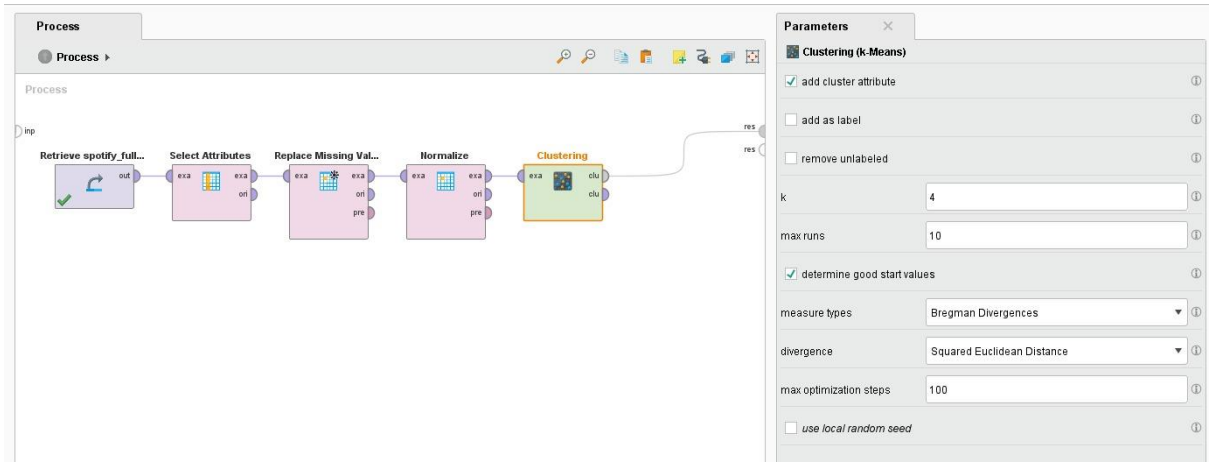
• Menggunakan metode **z-transformation** untuk melakukan standardisasi data.



5. K-Means Clustering

• Jumlah cluster (k) = 4

• Distance measure = Squared Euclidean Distance

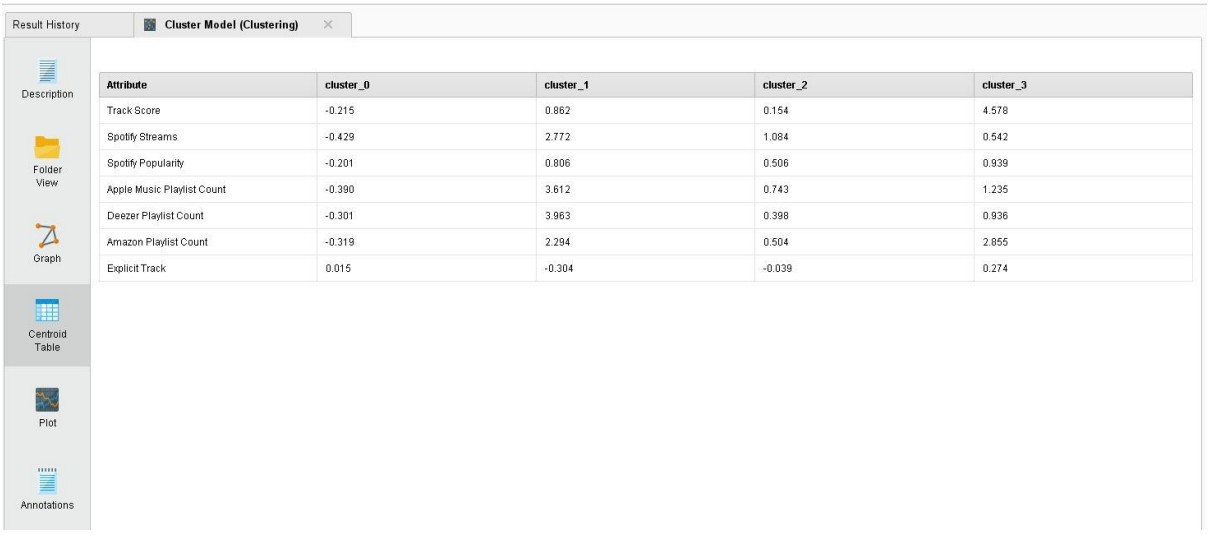


6. Hasil dan Visualisasi

• **Cluster Model**: Menampilkan jumlah item per cluster

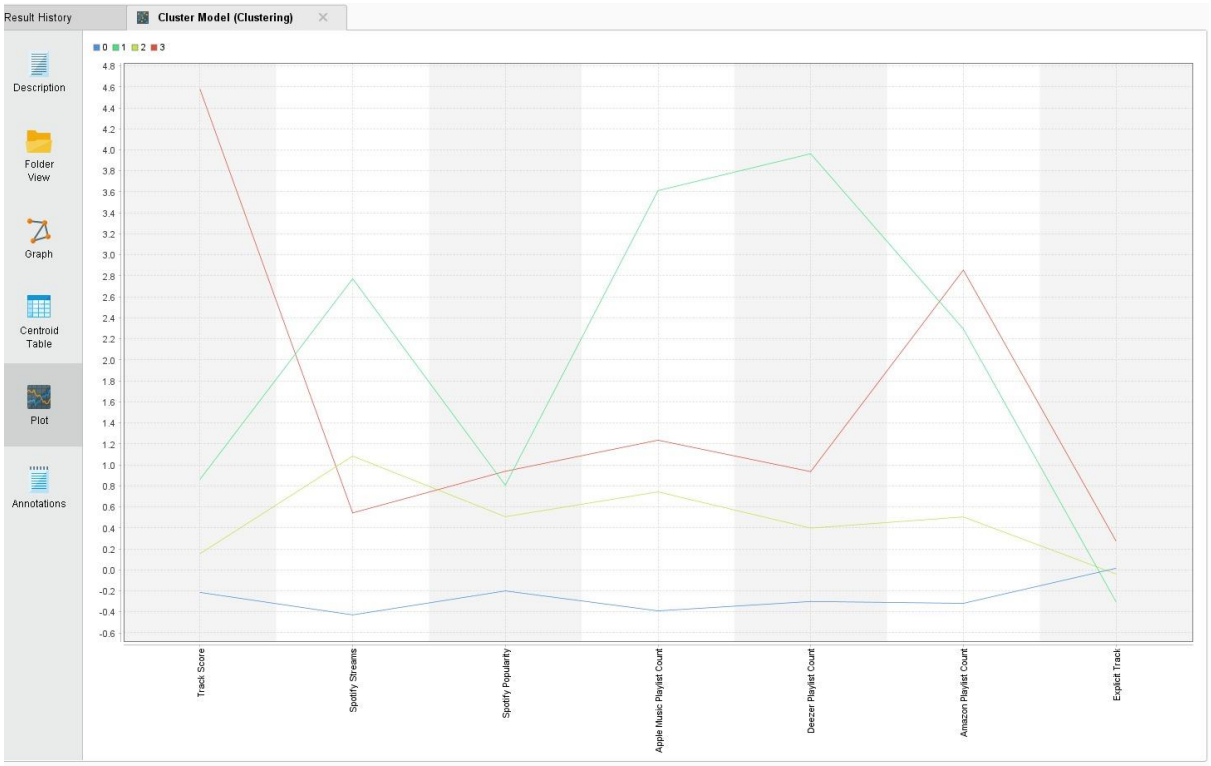
• **Graph**: Menampilkan bagaimana cluster terbagi

• **Centroid Table**: Menampilkan rata-rata nilai fitur untuk masing-masing cluster



Attribute	cluster_0	cluster_1	cluster_2	cluster_3
Track Score	-0.215	0.862	0.154	4.578
Spotify Streams	-0.429	2.772	1.084	0.542
Spotify Popularity	-0.201	0.806	0.506	0.939
Apple Music Playlist Count	-0.390	3.612	0.743	1.235
Deezer Playlist Count	-0.301	3.963	0.398	0.936
Amazon Playlist Count	-0.319	2.294	0.504	2.855
Explicit Track	0.015	-0.304	-0.039	0.274

• **Plot View**: Menampilkan grafik perbandingan antar cluster (misalnya line chart)


Result History
Cluster Model (Clustering)
Description
Folder View
Graph
Centroid Table
Plot
Annotations
0 1 2 3
4.8
4.6
4.4
4.2
4.0
3.8
3.6
3.4
3.2
3.0
2.8
2.6
2.4
2.2
2.0
1.8
1.6
1.4
1.2
1.0
0.8
0.6
0.4
0.2
0.0
-0.2
-0.4
-0.6
Track Score
Spotify Streams
Spotify Popularity
Apple Music Playlist Count
Deezer Playlist Count
Amazon Playlist Count
Explicit Track

4. Silahkan gunakan dataset public yang bisa digunakan untuk klasifikasi dengan menggunakan SVM. Adapun dataset tersebut:
   - Jumlah kelasnya minimal 3
   - Jumlah fiturnya (selain kelas minimal 7)
   - Jumlah total datanya minimal 1000.

   a. Tunjukkan data linknya dan lakukan EDA (Exploratory Data Analysis) dari dataset yang digunakan **(LO3) (5 point)**
   Dataset: Dry Bean Dataset
   Link Dataset: https://archive.ics.uci.edu/ml/datasets/dry+bean+dataset
   Link Google Colab (.ipynb):
   https://colab.research.google.com/drive/1YYrxyyqm4gng3IVZWKysotB1aDIakk4M?usp=sharing

   Spesifikasi Dataset:
   • **Jumlah data:** 13.611 sampel
   • **Jumlah fitur:** 16 fitur numerik (tanpa kolom ID)
   • **Target:** Class (7 jenis kacang: *Barbunya, Bombay, Cali, Dermason, Horoz, Seker, Sira*)

   Hasil EDA:

   • Tidak terdapat missing values.

```
# Tampilkan jumlah missing value per kolom
print("\nMissing values per column:")
print(df.isnull().sum())
```

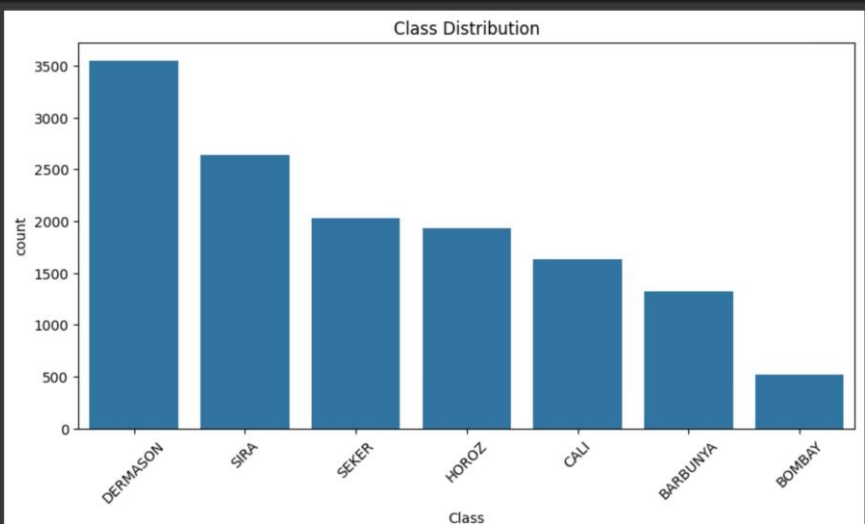
```
[4] # Tampilkan tipe data dan jumlah missing value
    df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 13611 entries, 0 to 13610
Data columns (total 17 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   Area             13611 non-null  int64
 1   Perimeter        13611 non-null  float64
 2   MajorAxisLength  13611 non-null  float64
 3   MinorAxisLength  13611 non-null  float64
 4   AspectRatio      13611 non-null  float64
 5   Eccentricity     13611 non-null  float64
 6   ConvexArea       13611 non-null  int64
 7   EquivDiameter    13611 non-null  float64
 8   Extent           13611 non-null  float64
 9   Solidity         13611 non-null  float64
 10  Roundness        13611 non-null  float64
 11  Compactness      13611 non-null  float64
 12  ShapeFactor1     13611 non-null  float64
 13  ShapeFactor2     13611 non-null  float64
 14  ShapeFactor3     13611 non-null  float64
 15  ShapeFactor4     13611 non-null  float64
 16  Class            13611 non-null  object
dtypes: float64(14), int64(2), object(1)
memory usage: 1.8+ MB
```

   • Semua fitur bertipe numerik dan memiliki skala berbeda → dilakukan

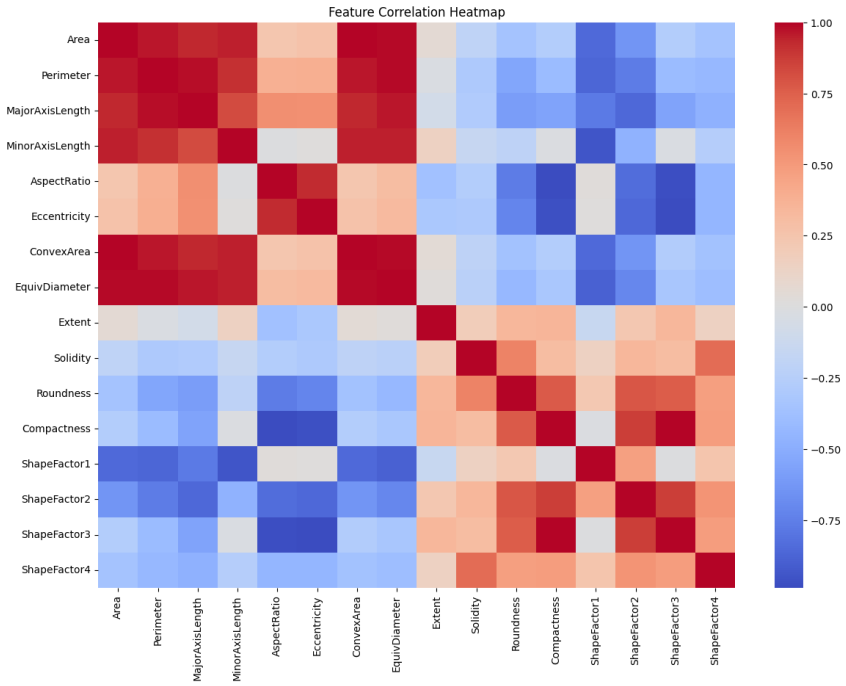
standardisasi.

• Distribusi kelas tidak seimbang: *Dermason* adalah mayoritas, *Bombay* paling sedikit. Jumlahnya Dermason (3546), SIRA (2636), SEKER(2027), HOROZ (1928), CALI (1630), BARBUNYA (1322), BOMBAY (522)

```
# Distribusi kelas (jumlah sampel per kelas)
plt.figure(figsize=(10,5))
sns.countplot(data=df, x='Class', order=df['Class'].value_counts().index)
plt.title("Class Distribution")
plt.xticks(rotation=45)
plt.show()

# Tampilkan jumlah absolut per kelas
print("Class value counts:\n", df['Class'].value_counts())
```



• Korelasi kuat antara fitur ukuran: Area, Perimeter, EquivDiameter, dsb.



b. Implementasikan dengan menggunakan salah 1 kernel functions SVM dan gunakan metric yang kalian anggap sesuai dan tariklah kesimpulan. **(LO3, LO4, LO5) (20 point)**

Tahapan:

1. Import Data

1. Library Imports and Dataset Loading

We begin by importing the necessary Python libraries and fetching the Dry Bean dataset directly from the UCI Machine Learning Repository using the `ucimlrepo` package. This package simplifies the dataset acquisition process by providing structured access to features, targets, and metadata.

```
[1] # Install ucimlrepo package (run only once in Colab)
    !pip install ucimlrepo
```

```
Requirement already satisfied: ucimlrepo in /usr/local/lib/python3.11/dist-packages (0.0.7)
Requirement already satisfied: pandas>=1.0.0 in /usr/local/lib/python3.11/dist-packages (from
Requirement already satisfied: certifi>=2020.12.5 in /usr/local/lib/python3.11/dist-packages (
Requirement already satisfied: numpy>=1.23.2 in /usr/local/lib/python3.11/dist-packages (from
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.11/dist-packag
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.11/dist-packages (from p
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.11/dist-packages (from
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.11/dist-packages (from pythc
```

```
[2] # Import required libraries
    import pandas as pd
    import numpy as np
    import matplotlib.pyplot as plt
    import seaborn as sns

    from ucimlrepo import fetch_ucirepo
```

```
# Fetch Dry Bean Dataset by ID (ID = 602)
dry_bean = fetch_ucirepo(id=602)

# Separate features and targets
X = dry_bean.data.features
y = dry_bean.data.targets

# Combine into one DataFrame for EDA
df = pd.concat([X, y], axis=1)

# Quick look
print("Dataset Shape:", df.shape)
df.head()
```

```
Dataset Shape: (13611, 17)
```

	Area	Perimeter	MajorAxisLength	MinorAxisLength	AspectRatio	Eccentricity	ConvexArea	EquivDiameter	Extent	Solidity	Roundness	Compactness	ShapeFactor1
0	28395	610.291	208.178117	173.888747	1.197191	0.549812	28715	190.141097	0.763923	0.988856	0.958027	0.913358	0.007332
1	28734	638.018	200.524796	182.734419	1.097356	0.411785	29172	191.272751	0.783968	0.984986	0.887034	0.953861	0.006979
2	29380	624.110	212.826130	175.931143	1.209713	0.562727	29690	193.410904	0.778113	0.989559	0.947849	0.908774	0.007244
3	30008	645.884	210.557999	182.516516	1.153638	0.498616	30724	195.467062	0.782681	0.976696	0.903936	0.928329	0.007017
4	30140	620.134	201.847882	190.279279	1.060798	0.333680	30417	195.896503	0.773098	0.990893	0.984877	0.970516	0.006697

2. Exploratory Data Analysis (EDA) Sudah dibahas di A
3. Data Preprocessing
   - Training 80% dan Test 20%
   - Label target diencode sehingga menjadi numerical values (0 to 6)

- Fiturnya di standarisasi

```
[9]  # Encode target labels (class names to integers)
     le = LabelEncoder()

     # Separate features and targets
     X = dry_bean.data.features
     y = le.fit_transform(dry_bean.data.targets['Class'])
```

```
[10] # Split the data (80% train, 20% test)
     X_train, X_test, y_train, y_test = train_test_split(
         X, y, test_size=0.2, stratify=y, random_state=42
     )
```

```
     # Standardize features (mean=0, std=1)
     scaler = StandardScaler()
     X_train_scaled = scaler.fit_transform(X_train)
     X_test_scaled = scaler.transform(X_test)
```

4. Model Training dengan 3 tahap yakni

- Model SVM Dasar

```
from sklearn.svm import SVC
from sklearn.model_selection import cross_val_score
from sklearn.metrics import accuracy_score, classification_report, ConfusionMatrixDispl

# Define SVM model (RBF kernel)
svm_model = SVC(kernel='rbf', C=1.0, gamma='scale', random_state=42)
```

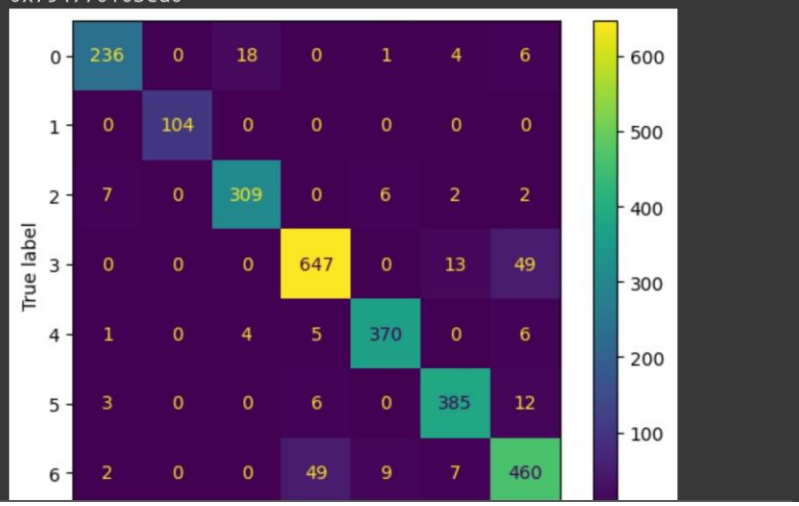
```
[13] # Cross-validation on training set
     cv_scores = cross_val_score(svm_model, X_train_scaled, y_train, cv=5, scoring='accuracy
     print(f"CV Accuracy: {cv_scores.mean():.4f} ± {cv_scores.std():.4f}")
```

```
CV Accuracy: 0.9313 ± 0.0048
```

```
[14] # Train on full training set and test on test set
     svm_model.fit(X_train_scaled, y_train)
     y_pred = svm_model.predict(X_test_scaled)
```

```
[15] # Evaluation
     print("\nTest Set Classification Report:")
     print(classification_report(y_test, y_pred))

     ConfusionMatrixDisplay.from_predictions(y_test, y_pred)
```

```
Test Set Classification Report:
              precision    recall  f1-score   support

           0       0.95      0.89      0.92       265
           1       1.00      1.00      1.00       104
           2       0.93      0.95      0.94       326
           3       0.92      0.91      0.91       709
           4       0.96      0.96      0.96       386
           5       0.94      0.95      0.94       406
           6       0.86      0.87      0.87       527

    accuracy                           0.92      2723
   macro avg       0.94      0.93      0.93      2723
weighted avg       0.92      0.92      0.92      2723

<sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at
0x794770103cd0>
```

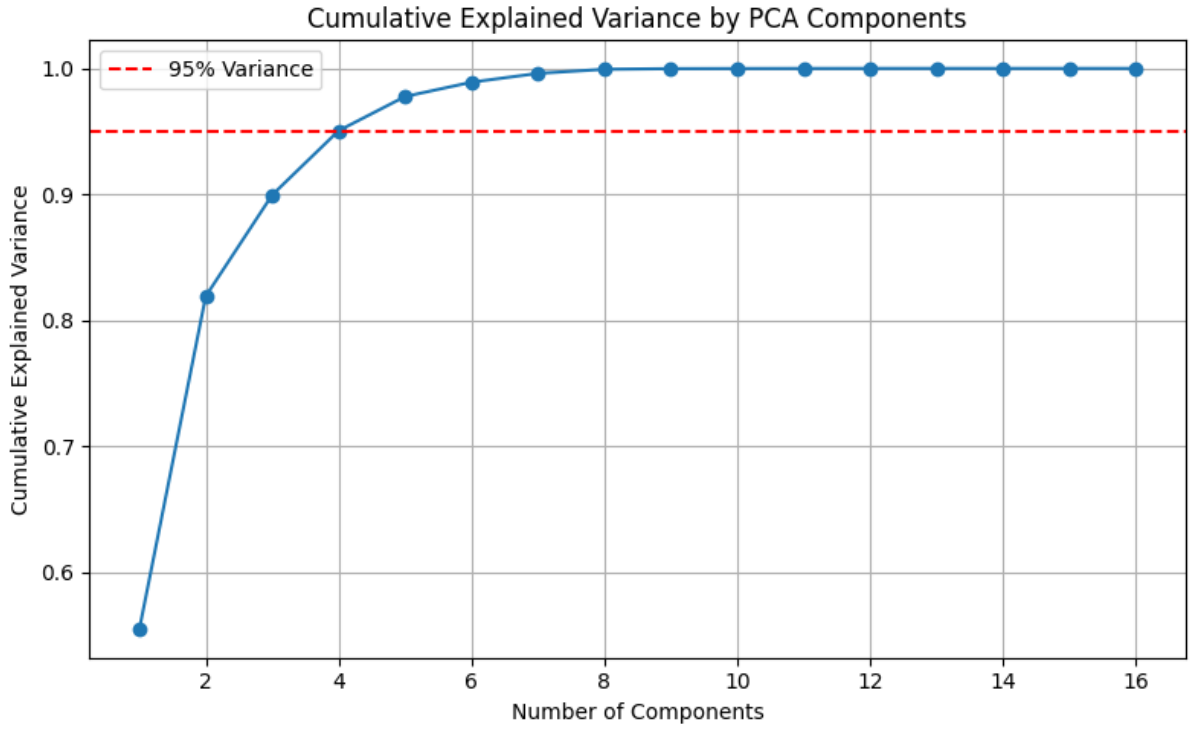


Model SVM pertama dilatih menggunakan parameter default dengan 16 fitur asli. Hasil evaluasi menunjukkan akurasi cross-validation sebesar 93,13% dan akurasi data uji sebesar 92%. Performa cukup baik di seluruh kelas, meskipun recall sedikit lebih rendah pada kelas minoritas seperti Bombay.

- PCA + SVM

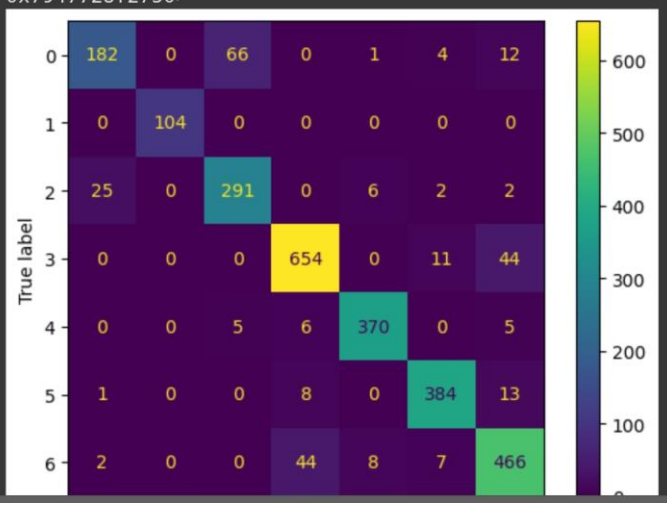
```python
from sklearn.decomposition import PCA
import numpy as np

# Apply PCA to training data only (fit on train, transform both train & test)
pca = PCA()
X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)
```

```python
# Plot cumulative explained variance
cum_var = np.cumsum(pca.explained_variance_ratio_)

plt.figure(figsize=(8, 5))
plt.plot(range(1, len(cum_var) + 1), cum_var, marker='o')
plt.axhline(y=0.95, color='r', linestyle='--', label='95% Variance')
plt.title('Cumulative Explained Variance by PCA Components')
plt.xlabel('Number of Components')
plt.ylabel('Cumulative Explained Variance')
plt.grid()
plt.legend()
plt.tight_layout()
plt.show()
```



```
[18] # Apply PCA with n_components that explains 95% of variance
     pca_95 = PCA(n_components=0.95)
     X_train_pca_95 = pca_95.fit_transform(X_train_scaled)
     X_test_pca_95 = pca_95.transform(X_test_scaled)

     # Cek berapa banyak komponen yang dipakai
     print(f"Number of components selected to reach 95% variance: {pca_95.n_components_}")

     Number of components selected to reach 95% variance: 4

[19] # Train SVM on PCA-reduced data
     svm_pca = SVC(kernel='rbf', C=1.0, gamma='scale', random_state=42)

[20] ! CV score
     rom sklearn.model_selection import cross_val_score
     v_scores_pca = cross_val_score(svm_pca, X_train_pca_95, y_train, cv=5, scoring='accurac
     rint(f"PCA+SVM CV Accuracy: {cv_scores_pca.mean():.4f} ± {cv_scores_pca.std():.4f}")

     PCA+SVM CV Accuracy: 0.8936 ± 0.0044

     # Final training and test evaluation
     svm_pca.fit(X_train_pca_95, y_train)
     y_pred_pca = svm_pca.predict(X_test_pca_95)

     from sklearn.metrics import classification_report, ConfusionMatrixDisplay

     print("Test Set Classification Report (after PCA):")
     print(classification_report(y_test, y_pred_pca, target_names=le.classes_))

     ConfusionMatrixDisplay.from_predictions(y_test, y_pred_pca)
```

```
Test Set Classification Report (after PCA):
              precision    recall  f1-score   support

     BARBUNYA       0.87      0.69      0.77       265
       BOMBAY       1.00      1.00      1.00       104
         CALI       0.80      0.89      0.85       326
     DERMASON       0.92      0.92      0.92       709
        HOROZ       0.96      0.96      0.96       386
        SEKER       0.94      0.95      0.94       406
         SIRA       0.86      0.88      0.87       527

     accuracy                           0.90      2723
    macro avg       0.91      0.90      0.90      2723
weighted avg       0.90      0.90      0.90      2723

<sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at
0x7947728f2750>
```



Selanjutnya dilakukan reduksi dimensi menggunakan PCA dengan mempertahankan 95% varians. Fitur berkurang dari 16 menjadi 4 komponen utama. Akurasi cross-validation menurun menjadi 89,36% dan akurasi data uji menjadi 90%. Walaupun sedikit menurun, kompleksitas model menjadi jauh lebih rendah.

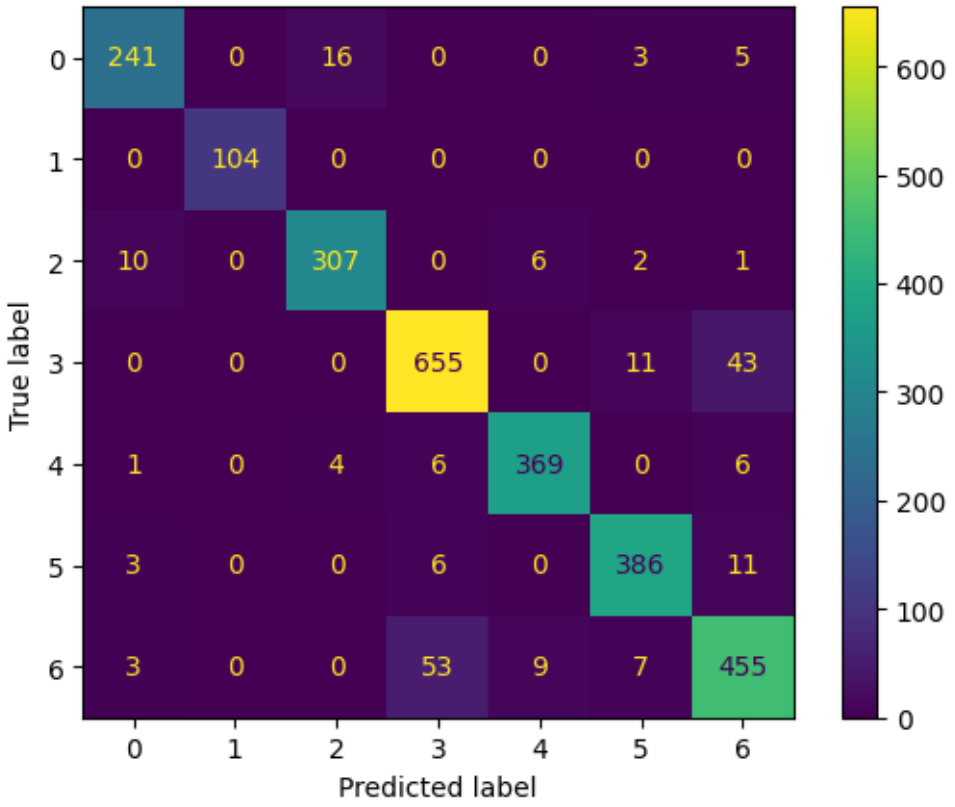
- SVM Tuning (Grid Search)

```
[25] from sklearn.model_selection import GridSearchCV
     from sklearn.svm import SVC

     # Define parameter grid
     param_grid = {
         'C': [0.1, 1, 10],
         'kernel': ['rbf', 'linear', 'poly'],
         'gamma': ['scale', 'auto']
     }

     # Setup GridSearchCV
     grid_search = GridSearchCV(
         estimator=SVC(),
         param_grid=param_grid,
         cv=5,
         scoring='accuracy',
         verbose=2,
         n_jobs=-1
     )

     # Fit on training data (tanpa PCA)
     grid_search.fit(X_train_scaled, y_train)

     # Best params and score
     print("Best Parameters:", grid_search.best_params_)
     print(f"Best Cross-Validation Accuracy: {grid_search.best_score_:.4f}")
```

```
Fitting 5 folds for each of 18 candidates, totalling 90 fits
Best Parameters: {'C': 10, 'gamma': 'scale', 'kernel': 'rbf'}
Best Cross-Validation Accuracy: 0.9331
```

```
[26] # Use best estimator from GridSearch
     best_svm = grid_search.best_estimator_

     # Predict on test set
     y_pred_tuned = best_svm.predict(X_test_scaled)

     from sklearn.metrics import classification_report, ConfusionMatrixDisplay

     # Evaluation
     print("Test Set Classification Report (Tuned SVM):")
     print(classification_report(y_test, y_pred_tuned, target_names=le.classes_))

     ConfusionMatrixDisplay.from_predictions(y_test, y_pred_tuned)
```

```
Test Set Classification Report (Tuned SVM):
              precision    recall  f1-score   support

    BARBUNYA       0.93      0.91      0.92       265
      BOMBAY       1.00      1.00      1.00       104
        CALI       0.94      0.94      0.94       326
    DERMASON       0.91      0.92      0.92       709
       HOROZ       0.96      0.96      0.96       386
       SEKER       0.94      0.95      0.95       406
        SIRA       0.87      0.86      0.87       527

    accuracy                           0.92      2723
   macro avg       0.94      0.94      0.94      2723
weighted avg       0.92      0.92      0.92      2723
```



Model SVM juga dituning menggunakan GridSearchCV untuk mencari parameter terbaik (C, kernel, gamma). Diperoleh parameter terbaik: C=10, kernel='rbf', gamma='scale'. Akurasi cross-validation meningkat menjadi 93,31%, dengan akurasi uji tetap di 92%. Model ini menunjukkan keseimbangan performa yang lebih baik antar kelas.

5. Kesimpulan

- Model SVM baseline sudah menunjukkan performa yang baik, dengan generalisasi yang kuat di seluruh kelas.
- Penerapan PCA membantu mengurangi dimensi dan menyederhanakan model, namun menyebabkan sedikit penurunan akurasi.
- Tuning SVM menggunakan GridSearchCV memberikan peningkatan performa, terutama pada kelas-kelas minoritas, dan menghasilkan model dengan keseimbangan prediksi yang paling baik.
- Rekomendasi akhir: gunakan model SVM hasil tuning dengan fitur lengkap dan parameter C=10, kernel='rbf', serta gamma='scale' untuk akurasi keseluruhan dan keseimbangan antar kelas yang optimal.

5. Jalankan script dari link berikut : Soal Clustering.ipynb - Colab. Ada beberapa hal yang dapat kalian ubah-ubah untuk dapat menjawab pertanyaan dibawah:
   a. Sample size,
   b. Jenis sample dan parameternya (blobs, noisy_circles, noisy_moons) cobalah ketiganya
   c. Parameter KMeans dan DBSCAN

   Silahkan run dan jawablah pertanyaan dibawah disertai bukti:

   Link Eksperimen : https://colab.research.google.com/drive/1qenS_SwGWBO-1V_b7zxRl2jc-Vc1zshL?usp=sharing

   a) Apakah DBSCAN akan selalu lebih baik dari pada KMeans? **(LO1) (5 point)**
   Tidak, DBSCAN tidak selalu lebih baik dari KMeans, Pada data yang memiliki bentuk cluster sederhana, simetris, dan bulat seperti pada dataset blobs, KMeans justru memberikan hasil yang lebih baik dan stabil. DBSCAN lebih unggul hanya pada bentuk cluster yang kompleks dan tidak linier.

   b) Apabila tidak, pada situasi apa sajikah DBSCAN tidak lebih baik dari KMeans ? **(LO1) (5 point)**

      1. Data memiliki distribusi cluster yang bulat dan terpisah rapi (contoh: blobs)
      2. Jika parameter eps tidak diatur dengan tepat, DBSCAN bisa menandai terlalu banyak titik sebagai noise (label = -1)
      3. Ukuran dataset sangat besar, sehingga proses DBSCAN menjadi lambat dan berat secara komputasi
      4. Data dengan kepadatan yang sangat berbeda antar cluster

   c) Di saat DBSCAN menghasilkan cluster yang lebih baik daripada KMeans, apakah KMeans dapat dioptimasi sehingga menghasilkan cluster yang sama baiknya dengan DBSCAN? **(LO2) (5 point)**
   Sebagian bisa, tapi terbatas. KMeans dapat ditingkatkan dengan:
      1. Menyesuaikan jumlah cluster (n_clusters)
      2. Menggunakan transformasi data (misalnya PCA atau metode kernel)
      3. Mengganti ke metode lain seperti Spectral Clustering

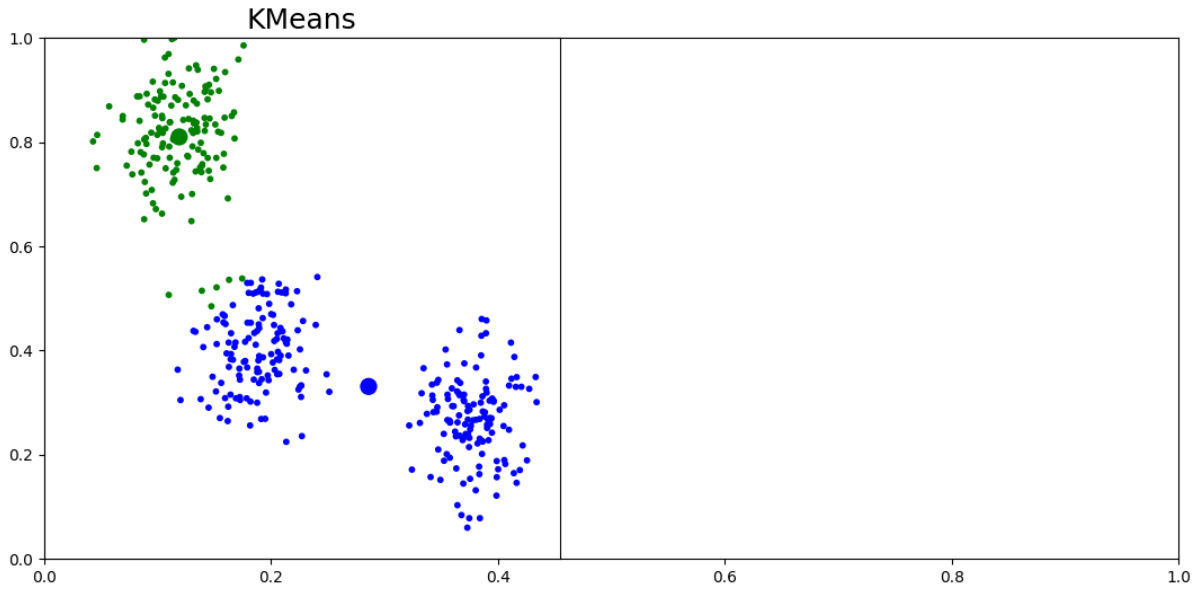
      Namun, karena KMeans mengasumsikan bentuk cluster yang bulat dan linier, tetap sulit menyamai DBSCAN untuk bentuk data kompleks seperti bulan sabit dan lingkaran.

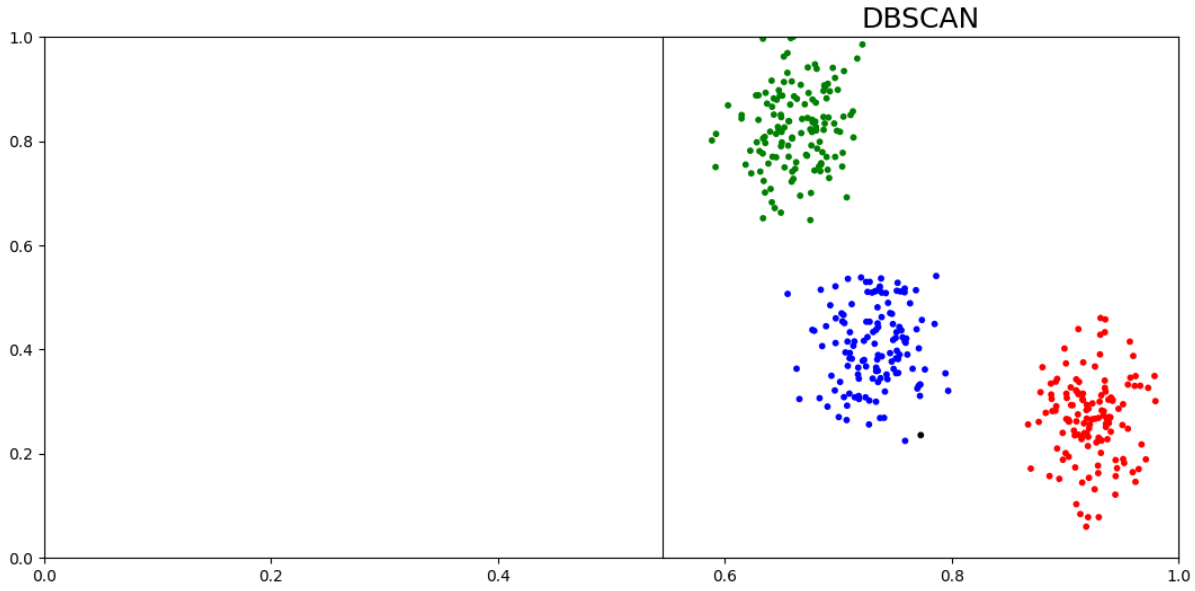
   d) Berikan kesimpulan yang dapat ditarik **(LO4) (10 point)**
   Berdasarkan eksperimen:

- KMeans cocok untuk data dengan distribusi cluster bulat dan terpisah dengan jelas.
- DBSCAN lebih unggul untuk data dengan bentuk yang kompleks, seperti lengkung atau struktur tidak beraturan.
- Tidak ada algoritma clustering yang paling baik untuk semua jenis data — pemilihan metode harus menyesuaikan bentuk dan struktur data.
- Parameterisasi sangat penting, terutama untuk DBSCAN (eps dan min_samples).
- Visualisasi hasil sangat membantu dalam mengevaluasi performa algoritma terhadap bentuk data.

Eksperimen 1 : Clustering dengan Blobs

```
[ ]  # Dataset: blobs
     n_samples = 400
     blobs = datasets.make_blobs(n_samples=n_samples, random_state=9)
     X, y = blobs
     X = StandardScaler().fit_transform(X)

     # Clustering Algorithms
     k_means = cluster.KMeans(n_clusters=2)
     dbscan = cluster.DBSCAN(eps=0.3)

     # Plot results
     plot_num = 1
     clustering_names = ['KMeans', 'DBSCAN']
     clustering_algorithms = [k_means, dbscan]

     for name, algorithm in zip(clustering_names, clustering_algorithms):
         algorithm.fit(X)
         y_pred = algorithm.labels_.astype(int) if hasattr(algorithm, 'labels_') else algorithm.predict(X)

         plt.subplots(figsize=(13, 6))
         plt.subplot(1, len(clustering_algorithms), plot_num)
         plt.title(name, size=18)
         plt.scatter(X[:, 0], X[:, 1], color=colors[y_pred].tolist(), s=10)
         if hasattr(algorithm, 'cluster_centers_'):
             centers = algorithm.cluster_centers_
             center_colors = colors[:len(centers)]
             plt.scatter(centers[:, 0], centers[:, 1], s=100, c=center_colors)

         plt.xlim(-2, 2)
         plt.ylim(-2, 2)
         plt.xticks(())
         plt.yticks(())
         plot_num += 1
     plt.show()
```



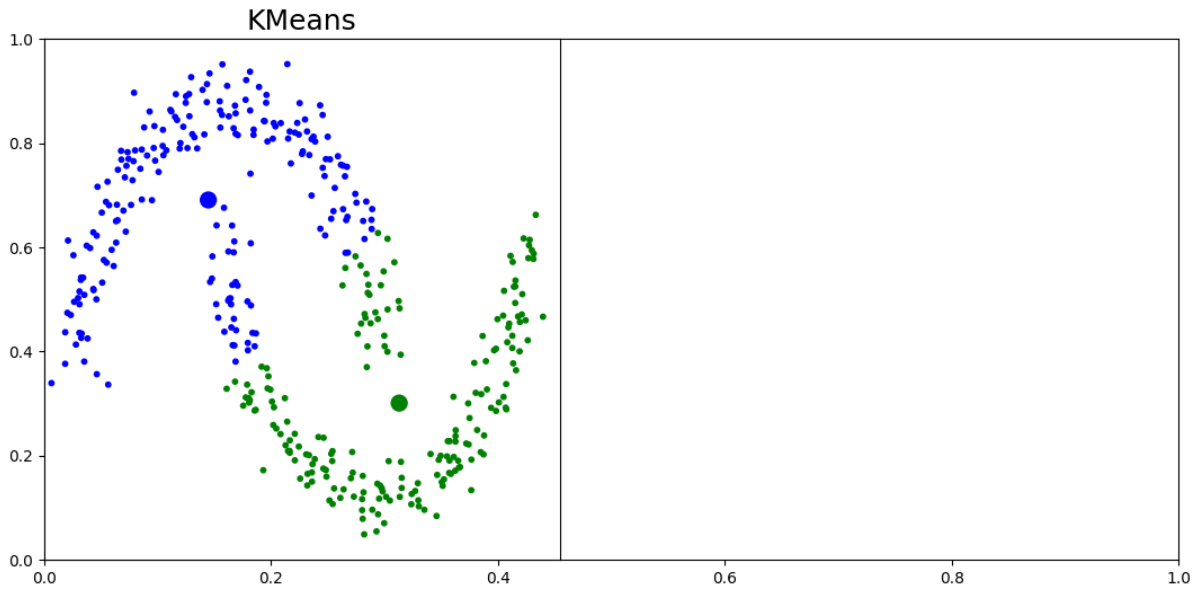


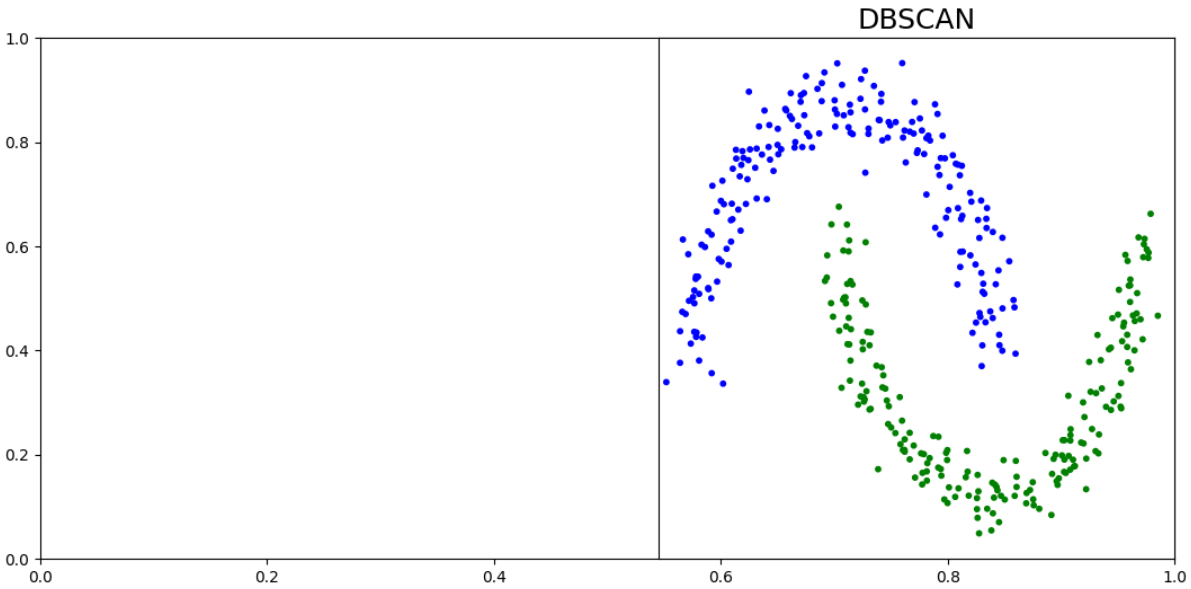
- KMeans: Berhasil memisahkan cluster dengan baik sesuai bentuk datanya.
- DBSCAN: Juga bisa mendeteksi cluster, tapi kadang ada titik dianggap noise jika parameter tidak pasKMeans berfungsi denga baik

Sehingga untuk kasus ini dengan data yang bentuknya bulat dan rapi lebih cocok menggunakan KMeans

Eksperimen 2: Clustering dengan noisy_moons

```
[ ]  # Dataset: noisy moons
     noisy_moons = datasets.make_moons(n_samples=n_samples, noise=0.08)
     X, y = noisy_moons
     X = StandardScaler().fit_transform(X)

     # Clustering Algorithms
     k_means = cluster.KMeans(n_clusters=2)
     dbscan = cluster.DBSCAN(eps=0.3)

     # Plot results
     plot_num = 1
     clustering_names = ['KMeans', 'DBSCAN']
     clustering_algorithms = [k_means, dbscan]

     for name, algorithm in zip(clustering_names, clustering_algorithms):
         algorithm.fit(X)
         y_pred = algorithm.labels_.astype(int) if hasattr(algorithm, 'labels_') else algorithm.predict(X)

         plt.subplots(figsize=(13, 6))
         plt.subplot(1, len(clustering_algorithms), plot_num)
         plt.title(name, size=18)
         plt.scatter(X[:, 0], X[:, 1], color=colors[y_pred].tolist(), s=10)
         if hasattr(algorithm, 'cluster_centers_'):
             centers = algorithm.cluster_centers_
             center_colors = colors[:len(centers)]
             plt.scatter(centers[:, 0], centers[:, 1], s=100, c=center_colors)

         plt.xlim(-2, 2)
         plt.ylim(-2, 2)
         plt.xticks(())
         plt.yticks(())
         plot_num += 1
     plt.show()
```



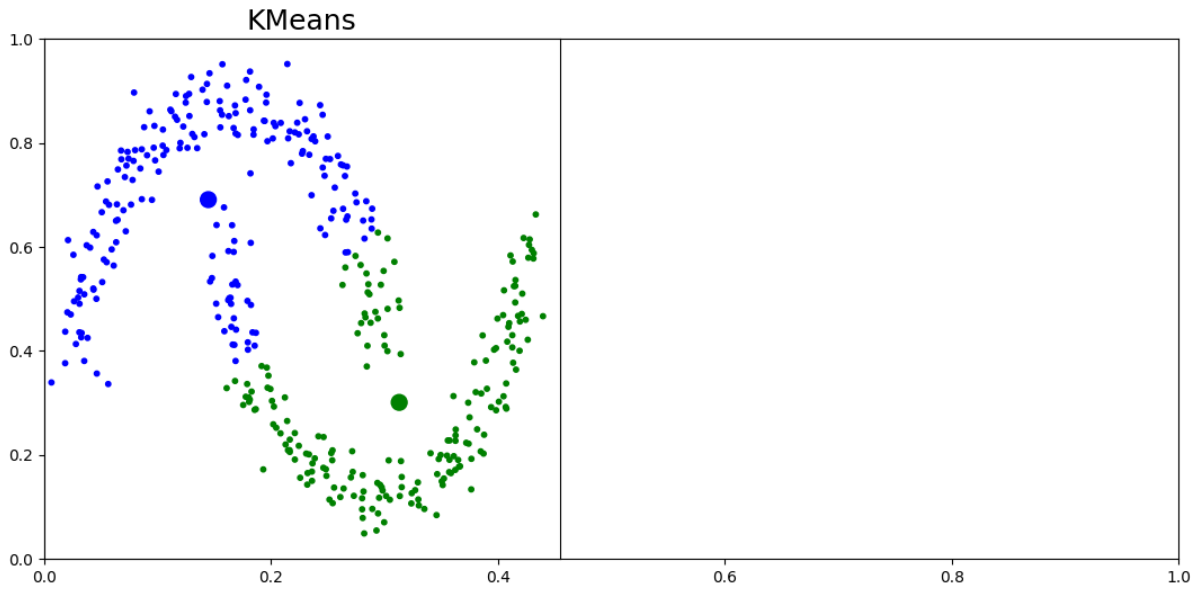


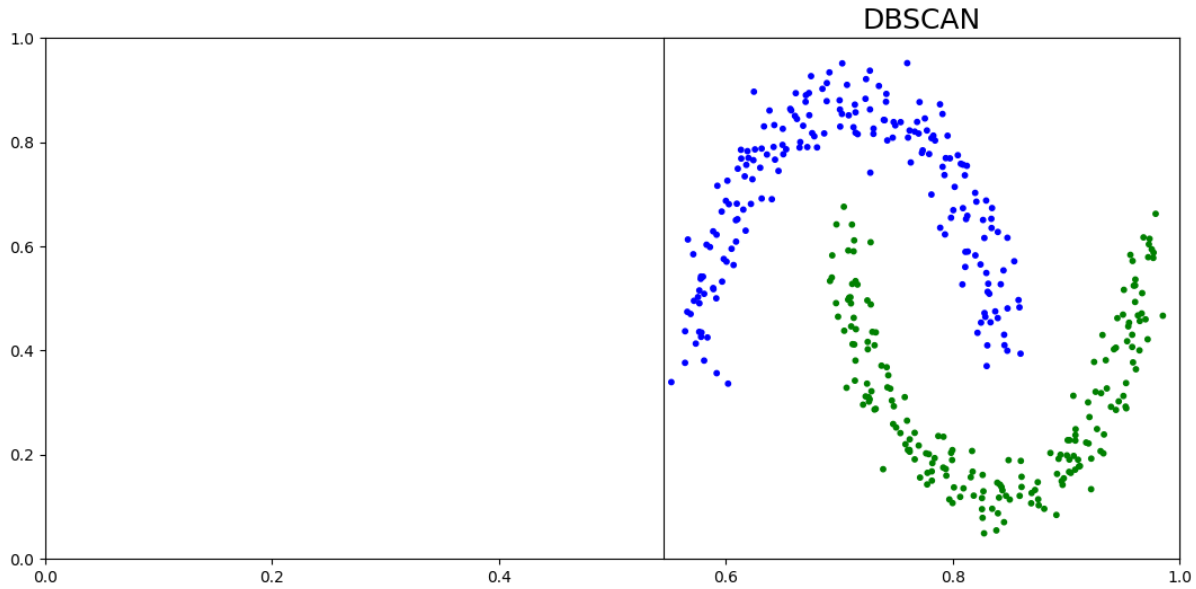
- KMeans: Gagal memisahkan dua bentuk bulan sabit karena hanya bisa memotong dengan garis lurus.
- DBSCAN: Mampu mengikuti bentuk bulan sabit dan memisahkan dengan baik.

Pada data dengan bentuk lengkung seperti ini, DBSCAN jauh lebih efektif daripada KMeans.

Eksperimen 3: Clustering dengan noisy_circles

```
[ ]  # Dataset: noisy moons
     noisy_moons = datasets.make_moons(n_samples=n_samples, noise=0.08)
     X, y = noisy_moons
     X = StandardScaler().fit_transform(X)

     # Clustering Algorithms
     k_means = cluster.KMeans(n_clusters=2)
     dbscan = cluster.DBSCAN(eps=0.3)

     # Plot results
     plot_num = 1
     clustering_names = ['KMeans', 'DBSCAN']
     clustering_algorithms = [k_means, dbscan]

     for name, algorithm in zip(clustering_names, clustering_algorithms):
         algorithm.fit(X)
         y_pred = algorithm.labels_.astype(int) if hasattr(algorithm, 'labels_') else algorithm.predict(X)

         plt.subplots(figsize=(13, 6))
         plt.subplot(1, len(clustering_algorithms), plot_num)
         plt.title(name, size=18)
         plt.scatter(X[:, 0], X[:, 1], color=colors[y_pred].tolist(), s=10)
         if hasattr(algorithm, 'cluster_centers_'):
             centers = algorithm.cluster_centers_
             center_colors = colors[:len(centers)]
             plt.scatter(centers[:, 0], centers[:, 1], s=100, c=center_colors)

         plt.xlim(-2, 2)
         plt.ylim(-2, 2)
         plt.xticks(())
         plt.yticks(())
         plot_num += 1
     plt.show()
```





- KMeans: Tidak bisa memisahkan dua lingkaran karena hanya bisa membagi secara lurus.
- DBSCAN: Berhasil memisahkan lingkaran luar dan dalam dengan baik.

DBSCAN sangat cocok untuk data berbentuk melingkar, sedangkan KMeans tidak mampu mengikuti bentuknya.